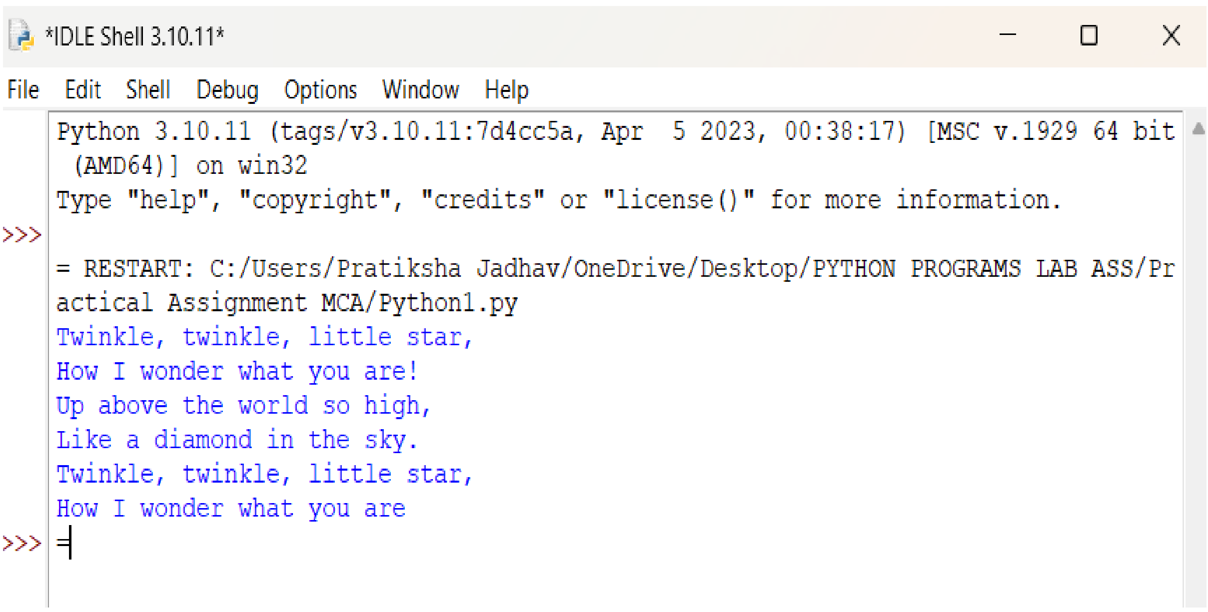
Assignment No : 1

Q1. Write a Python program to print the following string in a specific format ?

Out put :

Twinkle, twinkle, little star,
How I wonder what you are!
Up above the world so high,
Like a diamond in the sky.
Twinkle, twinkle, little star,
How I wonder what you are .

```
print("Twinkle, twinkle, little star,\n"
    "How I wonder what you are!\n"
    "Up above the world so high,\n"
    "Like a diamond in the sky.\n"
    "Twinkle, twinkle, little star,\n"
    "How I wonder what you are")
```



Q2. Write a program to perform arithmetic operations.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
add = a + b
sub = a - b
mul = a * b
div = a / b if b != 0 else "Undefined (division by zero)"
print("Addition:", add)
print("Subtraction:", sub)
print("Multiplication:", mul)
print("Division:", div)
```

IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Pr
actical Assignment MCA/Arithmetic operations.py
Enter first number: 24
Enter second number: 65
Addition: 89.0
Subtraction: -41.0
Multiplication: 1560.0
Division: 0.36923076923076925
>>>
```

Q3. Write a program using list to perform below operation:

a. Slicing

b. Concatenation

c. Repetition

```
lst1 = [1, 2, 3, 4, 5]
lst2 = ['a', 'b', 'c']
print("Slicing:", lst1[1:4])
print("Concatenation:", lst1 + lst2)
print("Repetition:", lst2 * 3)
```

IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Pr
actical Assignment MCA/List0peration.py
Slicing: [2, 3, 4]
Concatenation: [1, 2, 3, 4, 5, 'a', 'b', 'c']
Repetition: ['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c']
>>>
```

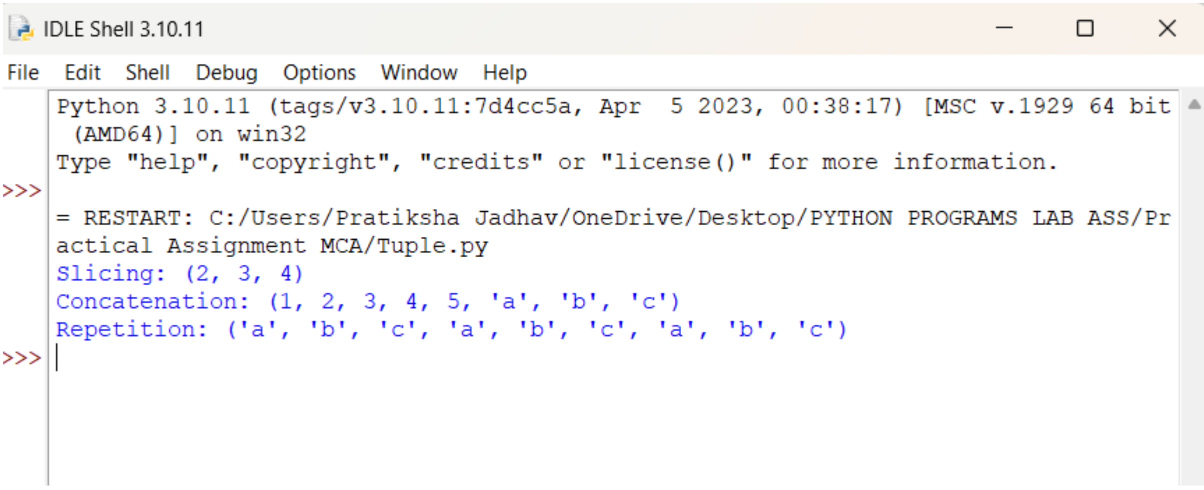
Q4. Write a program using tuple to perform below operation:

a. Slicing

b. Concatenation

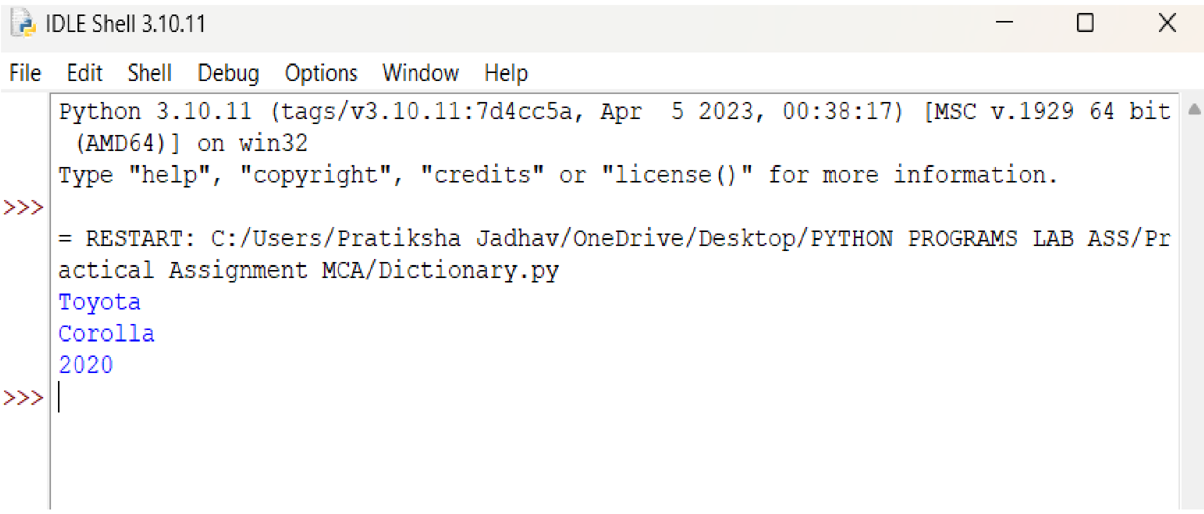
c. Repetition

```
t1 = (1, 2, 3, 4, 5)
t2 = ('a', 'b', 'c')
print("Slicing:", t1[1:4])
print("Concatenation:", t1 + t2)
print("Repetition:", t2 * 3)
```



Q5. Write a program using dictionary to access value using keys

```
car = {"brand": "Toyota", "model": "Corolla", "year": 2020}
print(car["brand"])
print(car["model"])
print(car["year"])
```



Q6. Write a program using set:

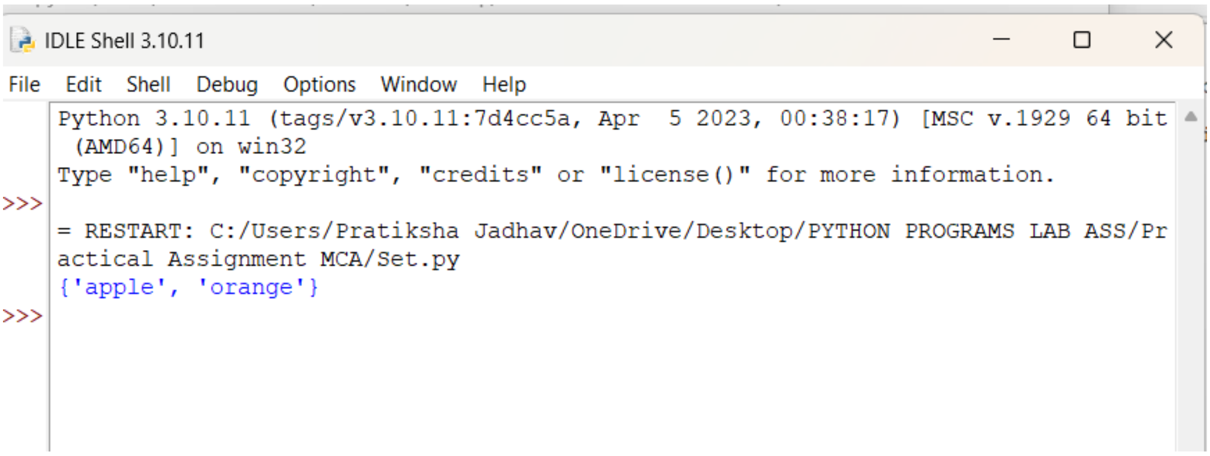
a. Create empty set

b. Assign value to set

c. Adding element

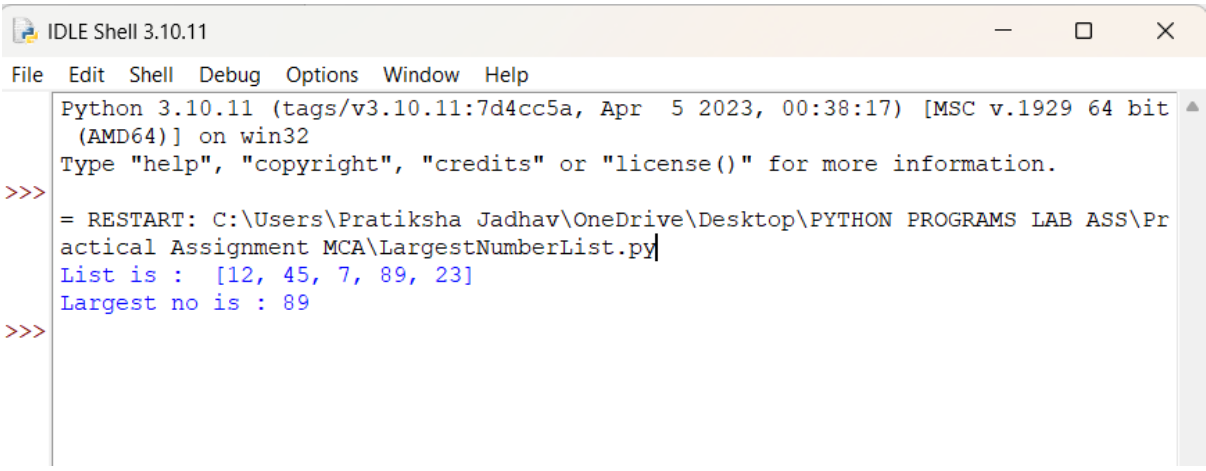
d. Removing element

```
s = set()
s = {"apple", "banana"}
s.add("orange")
s.remove("banana")
print(s)
```



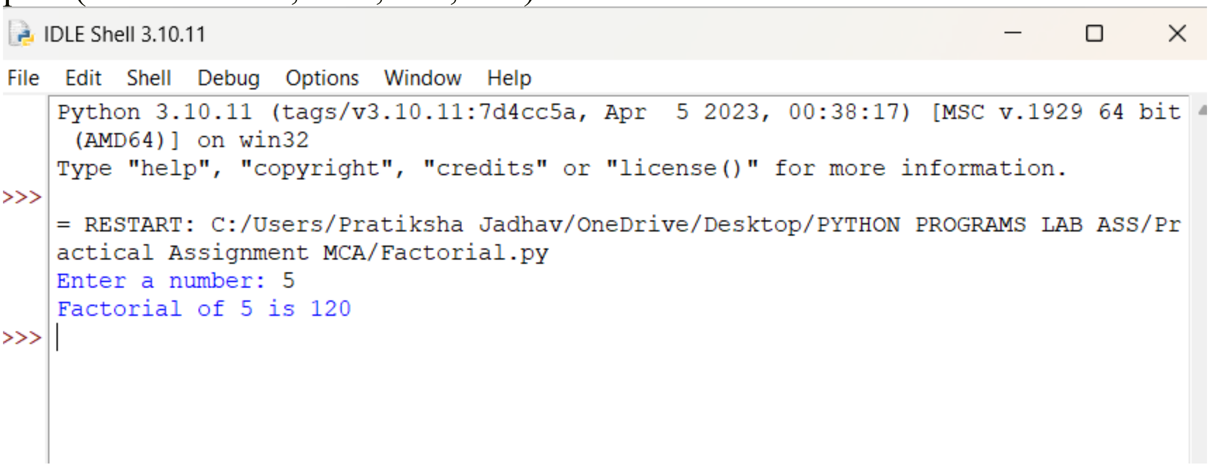
Q7. Write a program to get the largest number from the given list ?

```
numbers = [12, 45, 7, 89, 23]
print("List is : ",numbers)
print("Largest no is :",max(numbers))
```



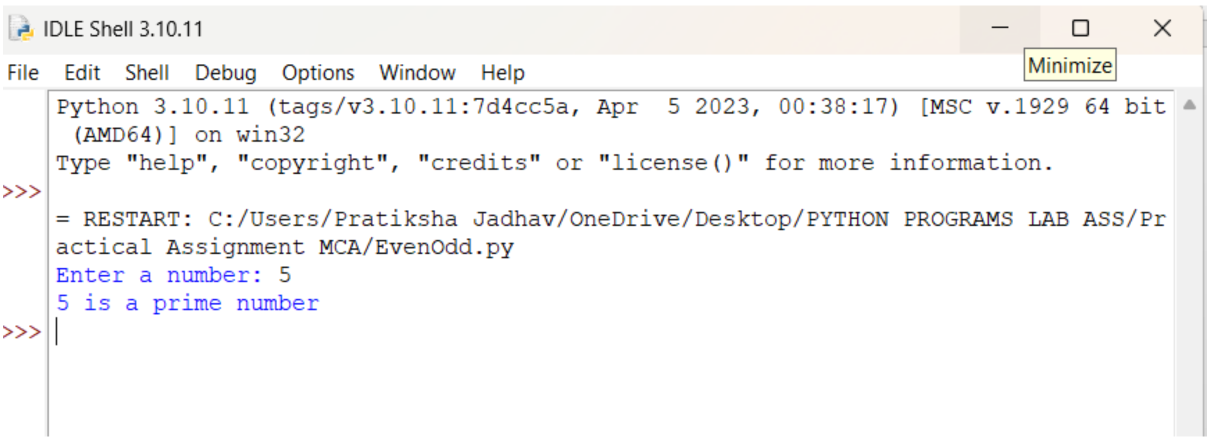
Q8. Write a program to find out Factorial of a given Number

```
num = int(input("Enter a number: "))
fact = 1
for i in range(1, num + 1):
    fact *= i
print("Factorial of", num, "is", fact)
```



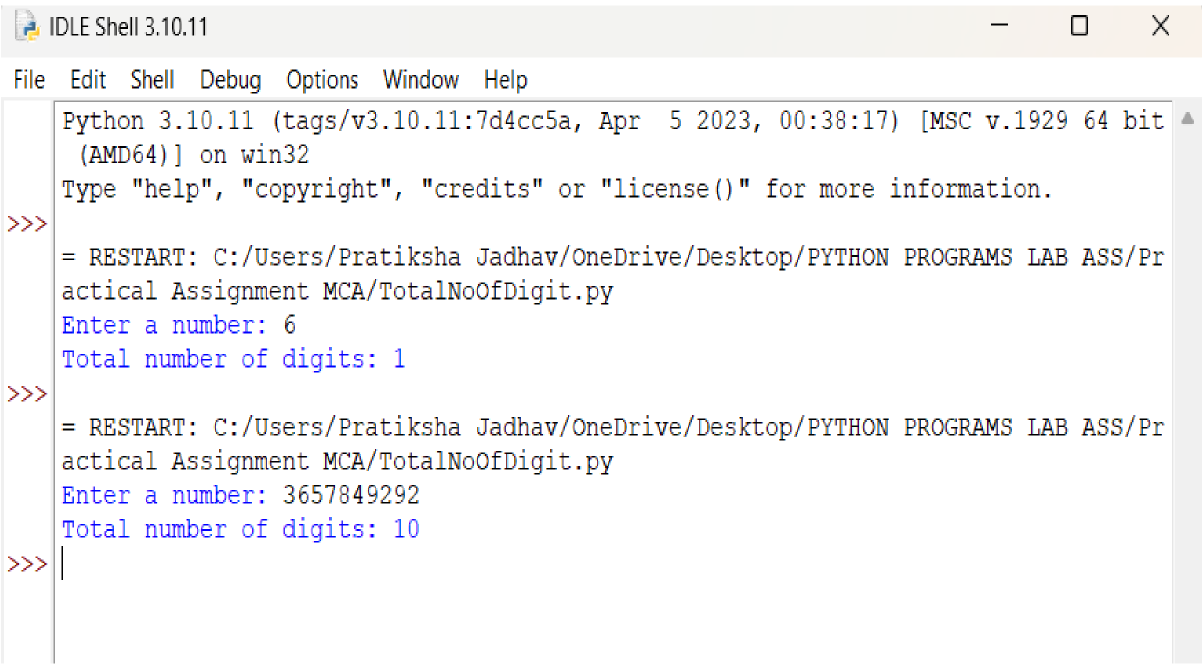
Q9. Write a program to find given number is prime or not.

```
num = int(input("Enter a number: "))
if num <= 1:
    print(num, "is not a prime number")
else:
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            print(num, "is not a prime number")
            break
    else:
        print(num, "is a prime number")
```



Q10. Write a program to identify total number of digits present in given number.

```
num = int(input("Enter a number: "))
count = 0
n = abs(num)   # handle negative numbers also

if n == 0:
    count = 1
else:
    while n > 0:
        n //= 10
        count += 1

print("Total number of digits:", count)
```



Q11. Write a program to display Fibonacci series up to a given number by using range function.

```
limit = int(input("Enter the limit: "))

a, b = 0, 1

print("Fibonacci series:")

for _ in range(limit):
    print(a, end=" ")
    a, b = b, a + b
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Fib
onacci Serier.py
Enter the limit: 99
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711
 28657 46368 75025 121393 196418 317811 514229 832040 1346269 2178309 3524578 57
02887 9227465 14930352 24157817 39088169 63245986 102334155 165580141 267914296 
433494437 701408733 1134903170 1836311903 2971215073 4807526976 7778742049 12586
269025 20365011074 32951280099 53316291173 86267571272 139583862445 225851433717
 365435296162 591286729879 956722026041 1548008755920 2504730781961 405273953788
1 6557470319842 10610209857723 17167680177565 27777890035288 44945570212853 7272
3460248141 117669030460994 190392490709135 308061521170129 498454011879264 80651
5533049393 1304969544928657 2111485077978050 3416454622906707 5527939700884757 8
944394323791464 14472334024676221 23416728348467685 37889062373143906 6130579072
1611591 99194853094755497 160500643816367088 259695496911122585 4201961407274896
73 679891637638612258 1100087778366101931 1779979416004714189 288006719437081612
0 4660046610375530309 7540113804746346429 12200160415121876738 19740274219868223
167 31940434634990099905 51680708854858323072 83621143489848422977 1353018523447
06746049
>>>
```

Q12. Write a program using for loop with:
String, Integer, Enumerate, Nested For,
List, Dictionary, Tuple, Zip()

```
# 1. For loop with String
print("\n--- For Loop with String ---")
text = "HELLO"
for ch in text:
    print(ch)

# 2. For loop with Integer (range)
print("\n--- For Loop with Integer using range() ---")
for i in range(1, 6):
    print(i)

# 3. For loop with Enumerate
print("\n--- For Loop with Enumerate ---")
fruits = ["Apple", "Banana", "Cherry"]
for index, item in enumerate(fruits):
    print(index, item)

# 4. Nested For Loop
print("\n--- Nested For Loop ---")
for i in range(1, 4):
    for j in range(1, 4):
        print(f"({i}, {j})")

# 5. For Loop with List
print("\n--- For Loop with List ---")
numbers = [10, 20, 30, 40]
for n in numbers:
    print(n)

# 6. For Loop with Dictionary
print("\n--- For Loop with Dictionary ---")
student = {"name": "John", "age": 20, "marks": 85}
for key, value in student.items():
    print(key, ":", value)

# 7. For Loop with Tuple
print("\n--- For Loop with Tuple ---")
t = (5, 10, 15, 20)
```

```
for item in t:
    print(item)

# 8. For Loop with zip()
print("\n--- For Loop with zip() ---")
names = ["John", "Alice", "Bob"]
scores = [85, 90, 78]
for n, s in zip(names, scores):
    print(n, "=>", s)
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Practical Assignment MCA/Forloop.py

--- For Loop with String ---
H
E
L
L
O

--- For Loop with Integer using range() ---
1
2
3
4
5

--- For Loop with Enumerate ---
0 Apple
1 Banana
2 Cherry

--- Nested For Loop ---
(1, 1)
(1, 2)
(1, 3)
(2, 1)
(2, 2)
(2, 3)
(3, 1)
(3, 2)
(3, 3)

--- For Loop with List ---
10
20
30
40

--- For Loop with Dictionary ---
name : John
age : 20
marks : 85

--- For Loop with Tuple ---
5
10
15
20

--- For Loop with zip() ---
John => 85
Alice => 90
Bob => 78
>>>
```

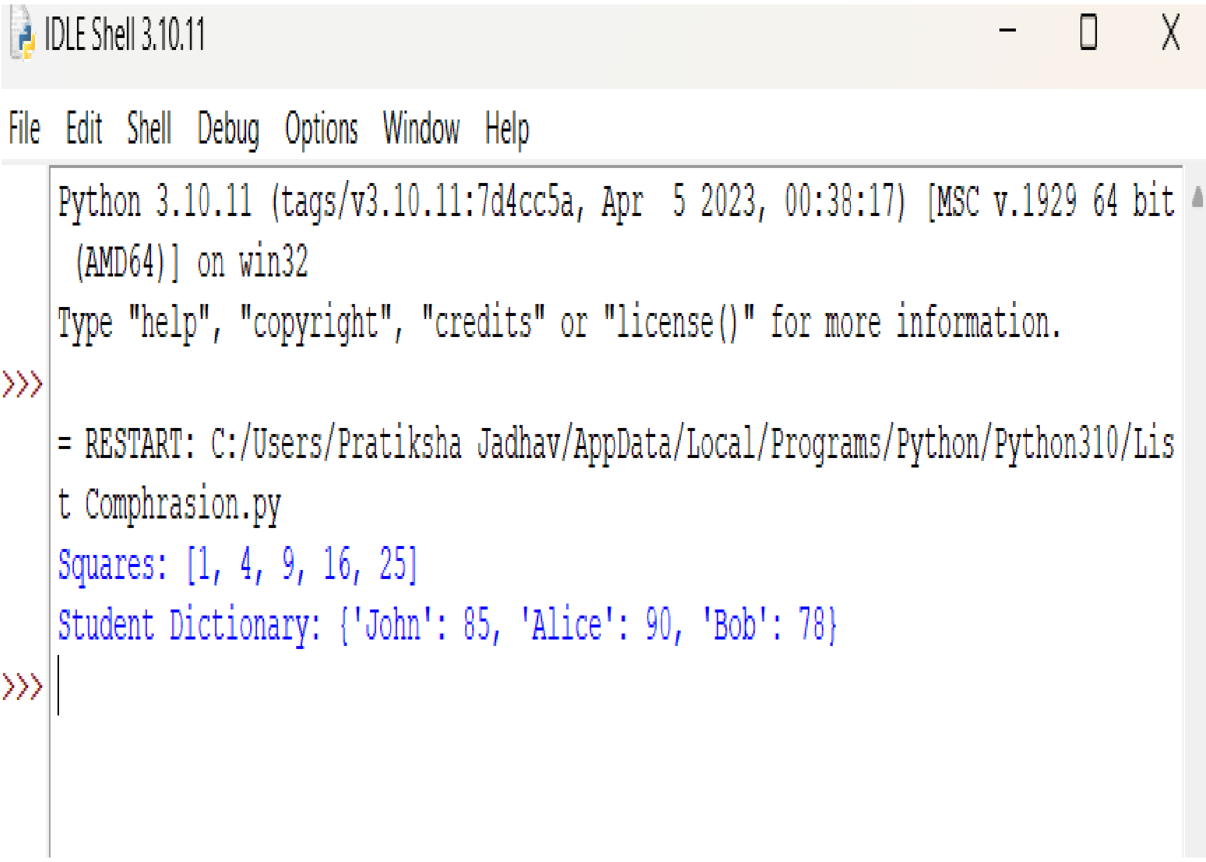
Q13. Write a program to perform Loop manipulation using pass, continue, break and else

```
print("\n--- Using pass ---")
for i in range(1, 6):
    if i == 3:
        pass  # pass does nothing, just a placeholder
    print("Value:", i)

print("\n--- Using continue ---")
for i in range(1, 6):
    if i == 3:
        continue  # skips printing 3 and moves to next iteration
    print("Value:", i)

print("\n--- Using break ---")
for i in range(1, 6):
    if i == 3:
        break  # loop stops completely when i = 3
    print("Value:", i)

print("\n--- Using else with loop ---")
for i in range(1, 6):
    print("Value:", i)
else:
    # This runs only when loop finishes normally (not stopped by break)
    print("Loop completed successfully!")
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
    Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
     (AMD64)] on win32
    Type "help", "copyright", "credits" or "license()" for more information.
>>>
    = RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Pr
    actical Assignment MCA/PassContinueBreak.py

    --- Using pass ---
    Value: 1
    Value: 2
    Value: 3
    Value: 4
    Value: 5

    --- Using continue ---
    Value: 1
    Value: 2
    Value: 4
    Value: 5

    --- Using break ---
    Value: 1
    Value: 2

    --- Using else with loop ---
    Value: 1
    Value: 2
    Value: 3
    Value: 4
    Value: 5
    Loop completed successfully!
>>> |
```

Q14. Write a program to perform Comprehensions on List Dictionaries.

```
# List comprehension
numbers = [1, 2, 3, 4, 5]
squares = [n*n for n in numbers]  # square of each number
print("Squares:", squares)

# Dictionary comprehension
students = ["John", "Alice", "Bob"]
marks = [85, 90, 78]

student_dict = {name: mark for name, mark in zip(students, marks)}  # map names to marks
print("Student Dictionary:", student_dict)
```



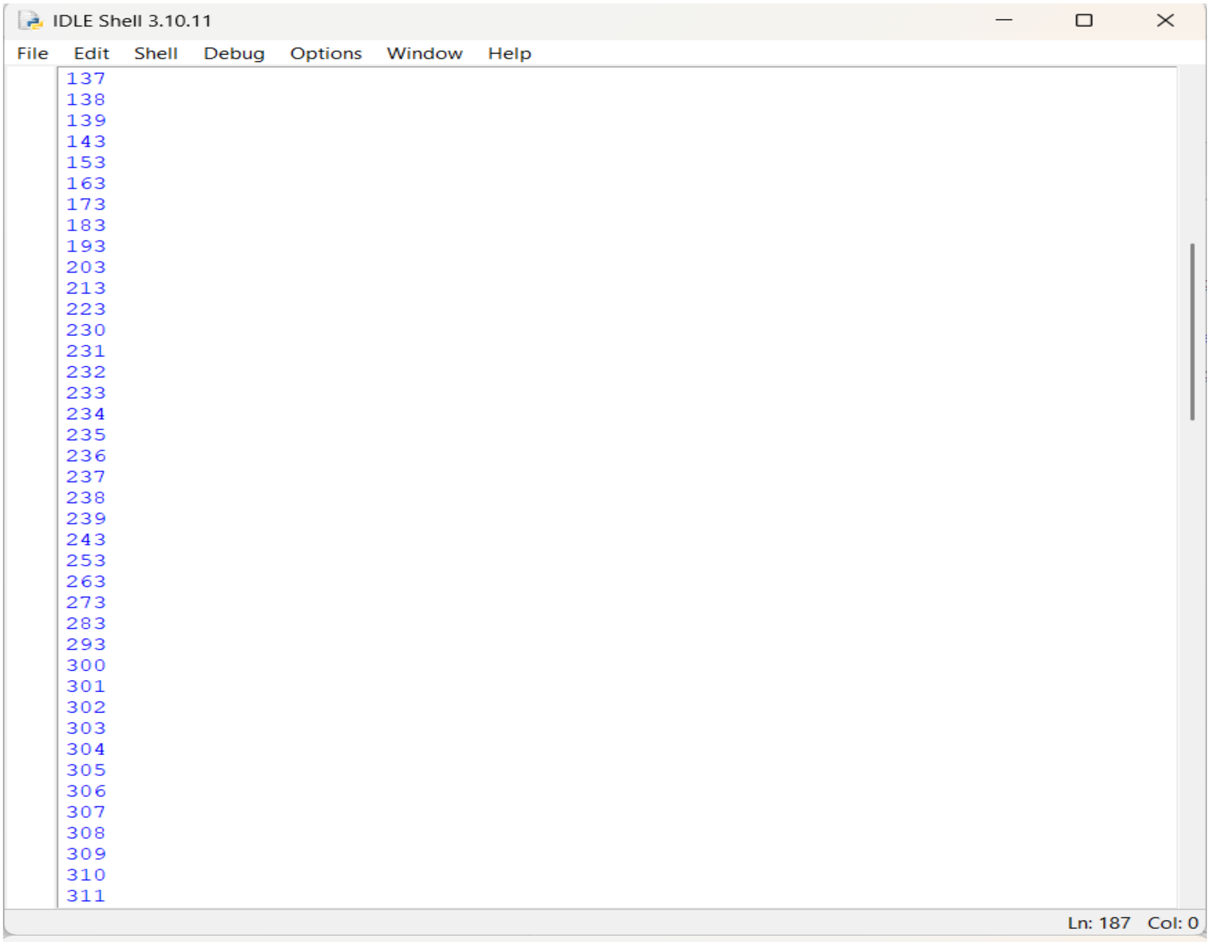
Q15. Write a program to Find all the numbers from 1-500 that have three in them.

```
# Print numbers from 1 to 500 that contain digit 3
for num in range(1, 501):
    if '3' in str(num):
        print(num)

# Store numbers in a list
nums_with_three = [num for num in range(1, 501) if '3' in str(num)]
print(nums_with_three)
```

IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Lis
t Comphrasion.py
Squares: [1, 4, 9, 16, 25]
Student Dictionary: {'John': 85, 'Alice': 90, 'Bob': 78}
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Num
bers.py
3
13
23
30
31
32
33
34
35
36
37
38
39
43
53
63
73
83
93
103
113
123
130
131
132
133
134
135
136
```

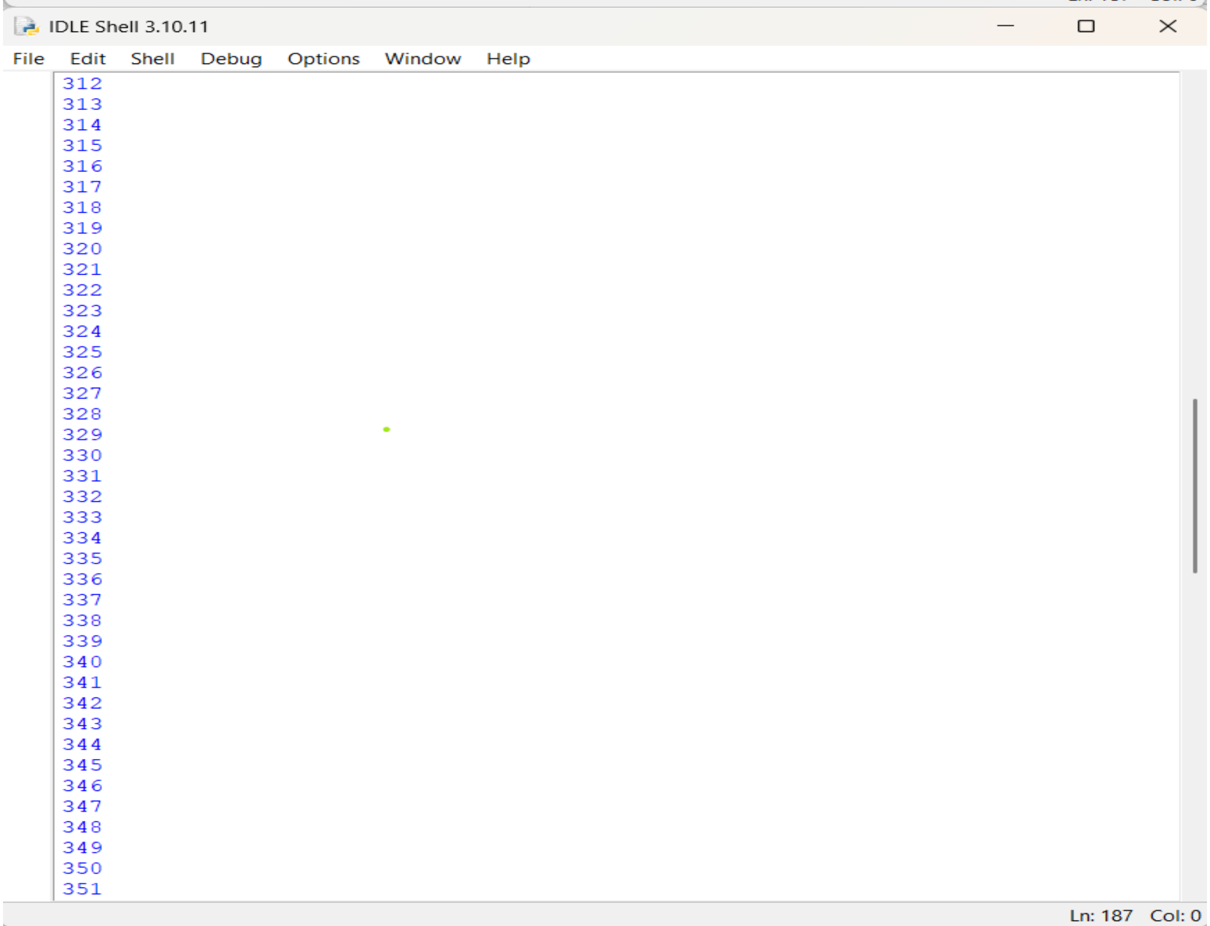


IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
137
138
139
143
153
163
173
183
193
203
213
223
230
231
232
233
234
235
236
237
238
239
243
253
263
273
283
293
300
301
302
303
304
305
306
307
308
309
310
311
```

Ln: 187 Col: 0



IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
```

Ln: 187 Col: 0

IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
```

Ln: 164 Col: 3

IDLE Shell 3.10.11

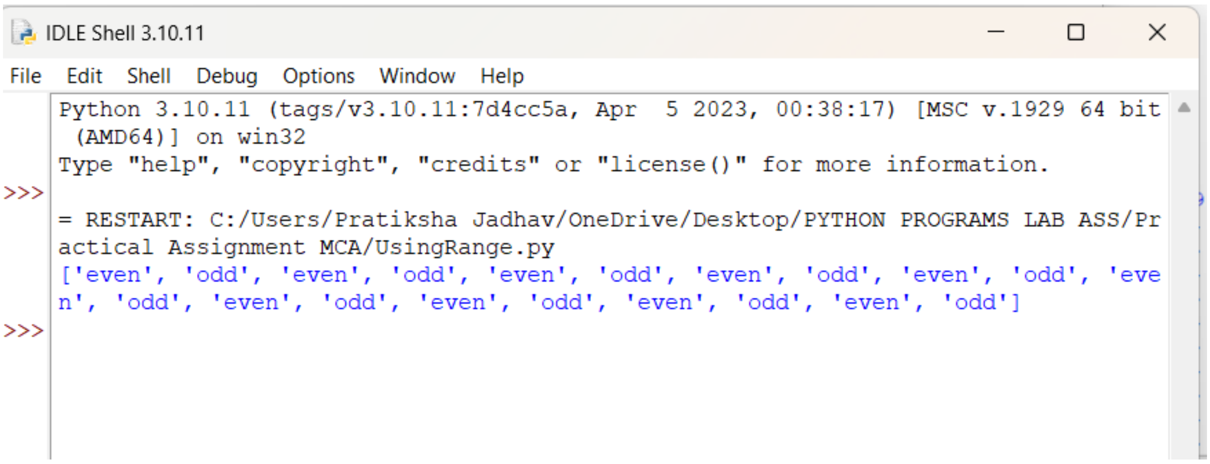
File Edit Shell Debug Options Window Help

```
390
391
392
393
394
395
396
397
398
399
403
413
423
430
431
432
433
434
435
436
437
438
439
443
453
463
473
483
493
[3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43, 53, 63, 73, 83, 93, 103,
 113, 123, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 143, 153, 163, 173,
 183, 193, 203, 213, 223, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 243,
 253, 263, 273, 283, 293, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310,
 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326,
 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342,
 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358,
 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374,
 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390,
 391, 392, 393, 394, 395, 396, 397, 398, 399, 403, 413, 423, 430, 431, 432, 433,
 434, 435, 436, 437, 438, 439, 443, 453, 463, 473, 483, 493]
```

Ln: 164 Col: 3

Q16 . Write a python program for numbers=range(20) produce a list containing the word "even" if the number is even or the word "odd" if the number is odd

```
numbers = range(20)
result = ["even" if n % 2 == 0 else "odd" for n in numbers]
print(result)
```

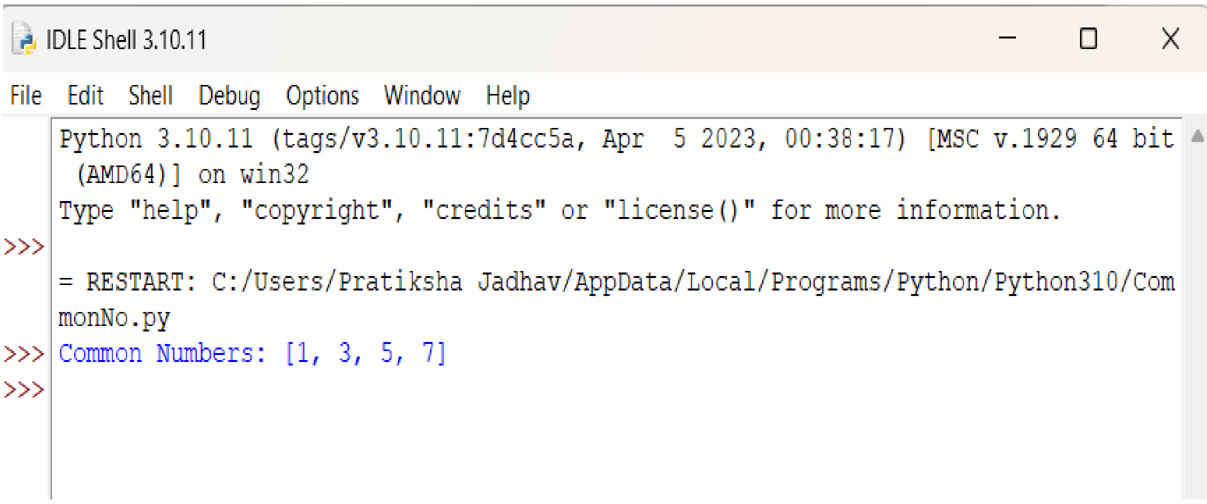


Q17. Find the common numbers in two list

1[1,3,4,5,6,7]

2[5,2,8,7,1,3]

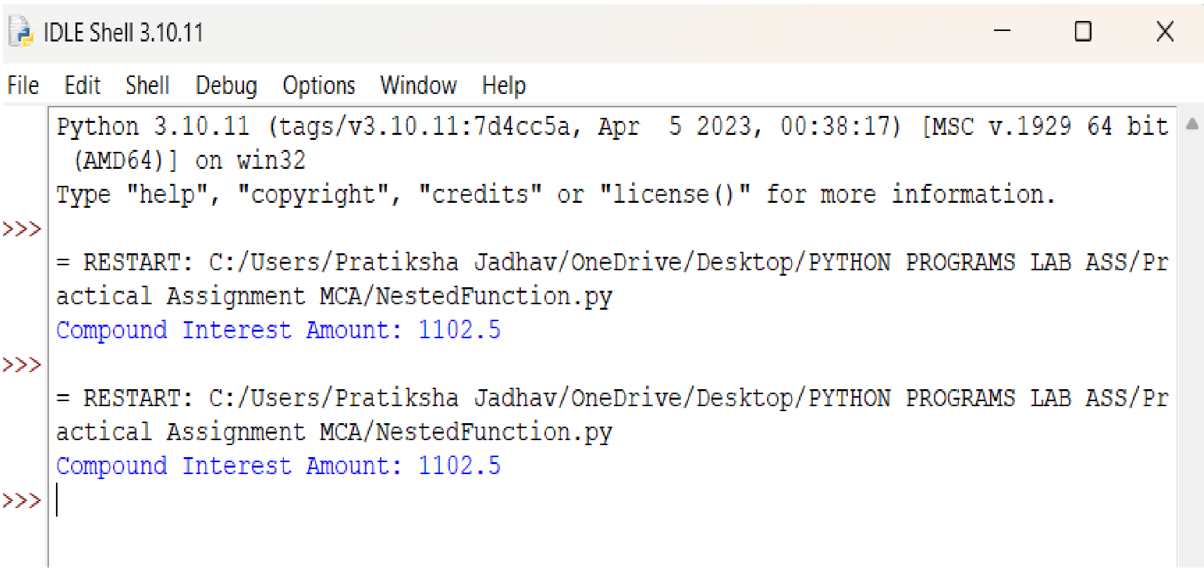
```
list1 = [1, 3, 4, 5, 6, 7]
list2 = [5, 2, 8, 7, 1, 3]
common = [num for num in list1 if num in list2]
print("Common Numbers:", common)
```



Assignment No : 2

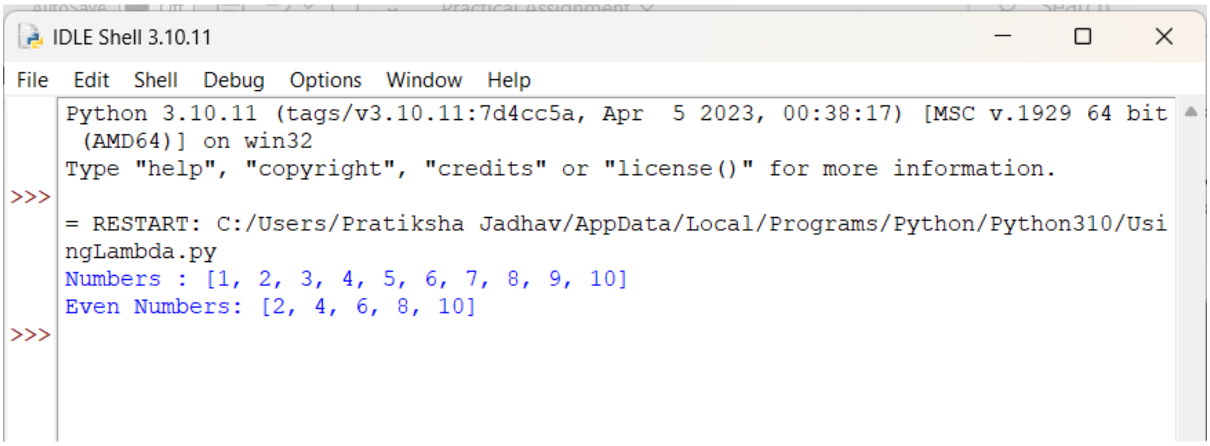
Q1. Implement a nested function that calculates compound interest using non-local variables

```
def compound_interest():
    principal = 0
    rate = 0
    time = 0

    def set_values(p, r, t):
        nonlocal principal, rate, time
        principal = p
        rate = r
        time = t
    def calculate():
        amount = principal * (1 + rate/100) ** time
        return amount
    set_values(1000, 5, 2)  # Example: P=1000, R=5%, T=2 years
    return calculate()

result = compound_interest()
print("Compound Interest Amount:", result)
```



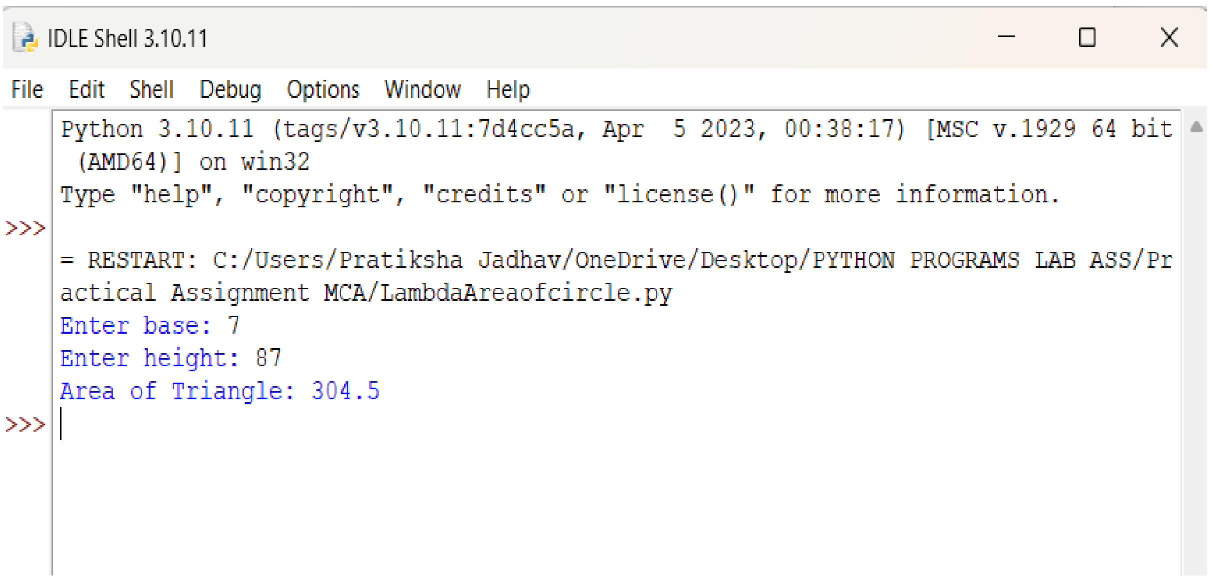
Q2. Write a program using lambda to filter even numbers from a list

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print("Numbers :",numbers)
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print("Even Numbers:", even_numbers)
```



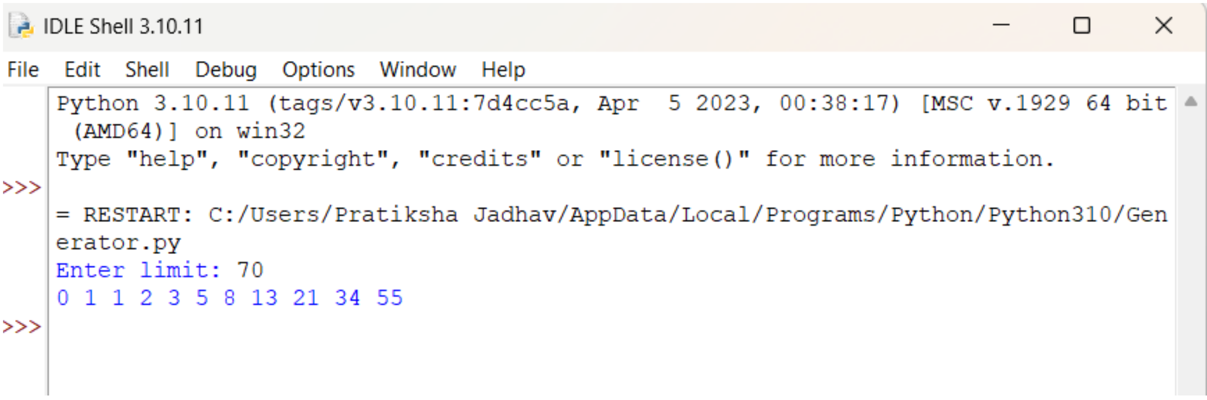
Q3. Write a program using lambda to find area of triangle

```
area = lambda base, height: 0.5 * base * height
b = float(input("Enter base: "))
h = float(input("Enter height: "))
print("Area of Triangle:", area(b, h))
```



Q4. Create a generator function to yield Fibonacci numbers up to a certain limit

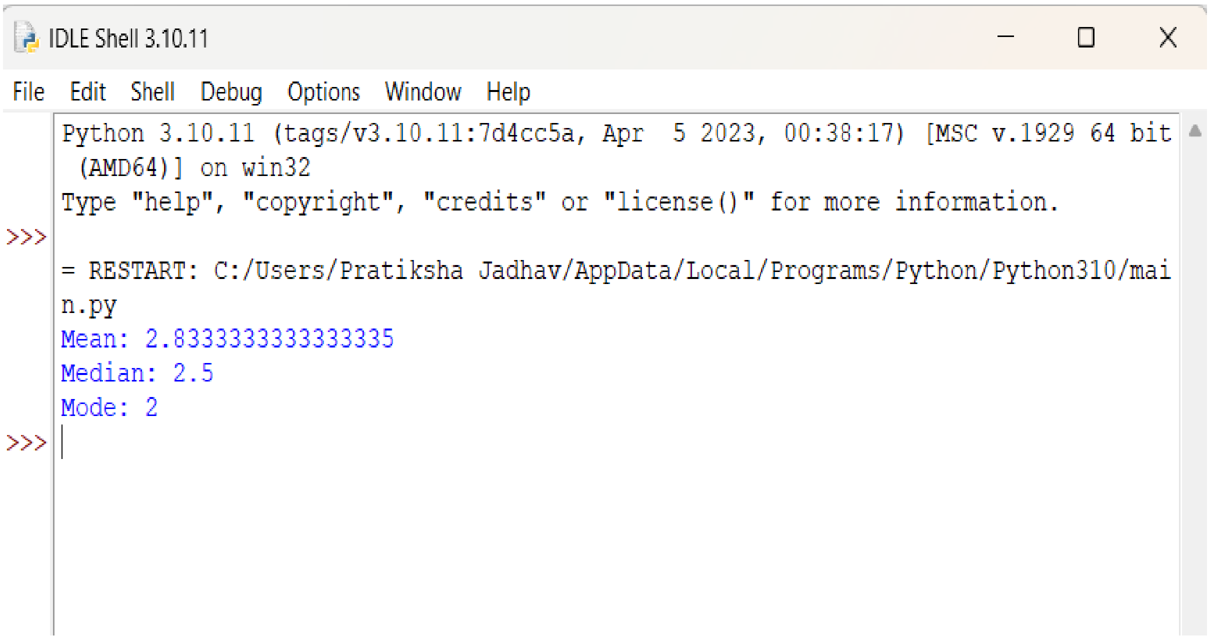
```
def fibonacci(limit):
    a, b = 0, 1
    while a <= limit:
        yield a
        a, b = b, a + b
limit = int(input("Enter limit: "))
for num in fibonacci(limit):
    print(num, end=" ")
```



Q5. Develop a custom Python module with functions for basic statistical operations (mean,median,mode) .

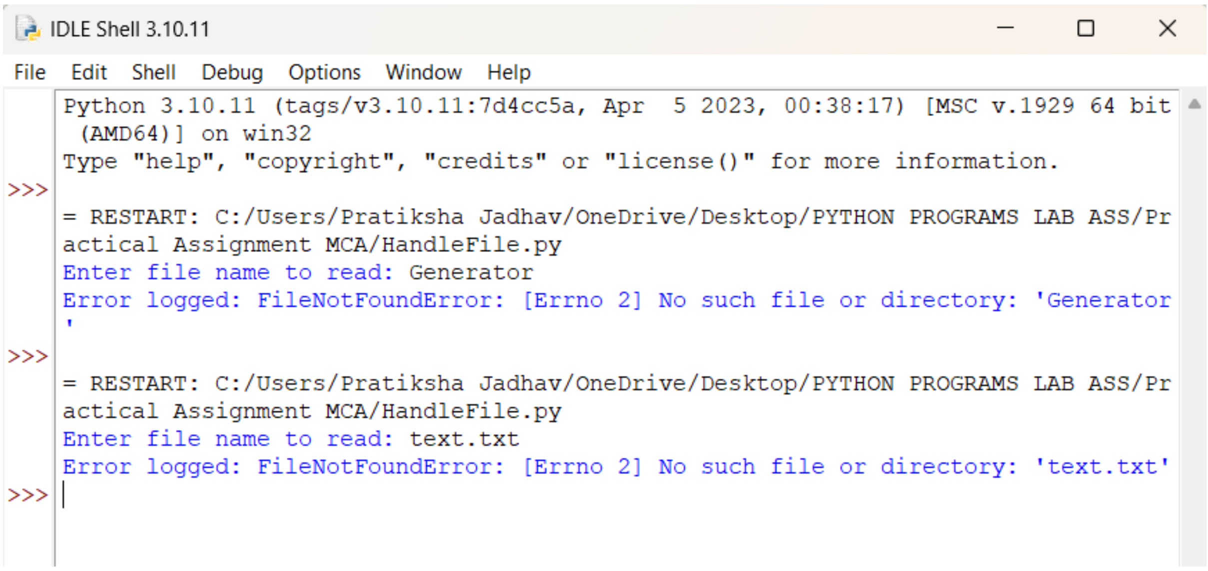
```
# mystats.py
# Custom module for basic statistical operations

def mean(data):
    return sum(data) / len(data)

def median(data):
    sorted_data = sorted(data)
    n = len(sorted_data)

    mid = n // 2
    if n % 2 == 0:
        return (sorted_data[mid - 1] + sorted_data[mid]) / 2
```

```
    else:
        return sorted_data[mid]

def mode(data):
    frequency = {}
    for num in data:
        frequency[num] = frequency.get(num, 1) + 1
    max_count = max(frequency.values())
    modes = [key for key, value in frequency.items() if value == max_count]

    if len(modes) == 1:
        return modes[0]
    else:
        return modes  # multiple modes


# main.py

import mystats
numbers = [1, 2, 2, 3, 4, 5]
print("Mean:", mystats.mean(numbers))
print("Median:", mystats.median(numbers))
print("Mode:", mystats.mode(numbers))
```



Q6. Write a program to handle file reading errors and log them to a separate file

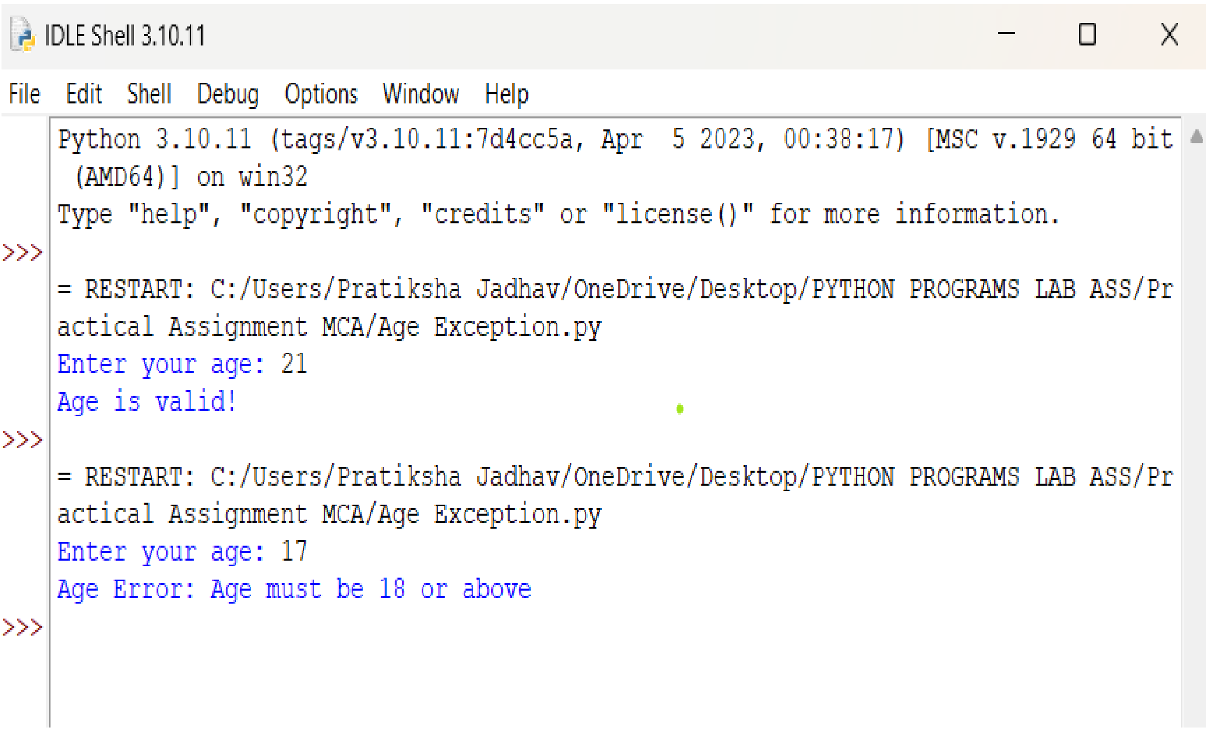
```
def read_file(filename):
    try:
        with open(filename, "r") as file:
            data = file.read()
            print("File Content:\n", data)

    except FileNotFoundError as e:
        log_error(f"FileNotFoundError: {e}")

    except PermissionError as e:
        log_error(f"PermissionError: {e}")

    except Exception as e:
        log_error(f"Other Error: {e}")

def log_error(message):
    with open("error_log.txt", "a") as log:
        log.write(message + "\n")
    print("Error logged:", message)

filename = input("Enter file name to read: ")
read_file(filename)
```



```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Pr
actical Assignment MCA/HandleFile.py
Enter file name to read: Generator
Error logged: FileNotFoundError: [Errno 2] No such file or directory: 'Generator
'
>>>
= RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Pr
actical Assignment MCA/HandleFile.py
Enter file name to read: text.txt
Error logged: FileNotFoundError: [Errno 2] No such file or directory: 'text.txt'
>>>
```

Q7. Create a custom exception class AgeException and use it to validate user input

```
class AgeException(Exception):
    pass  # custom exception class
try:
    age = int(input("Enter your age: "))
    if age < 0:
        raise AgeException("Age cannot be negative")
    elif age < 18:
        raise AgeException("Age must be 18 or above")
    print("Age is valid!")
except AgeException as e:
    print("Age Error:", e)

except ValueError:
    print("Invalid input! Please enter a number.")
```

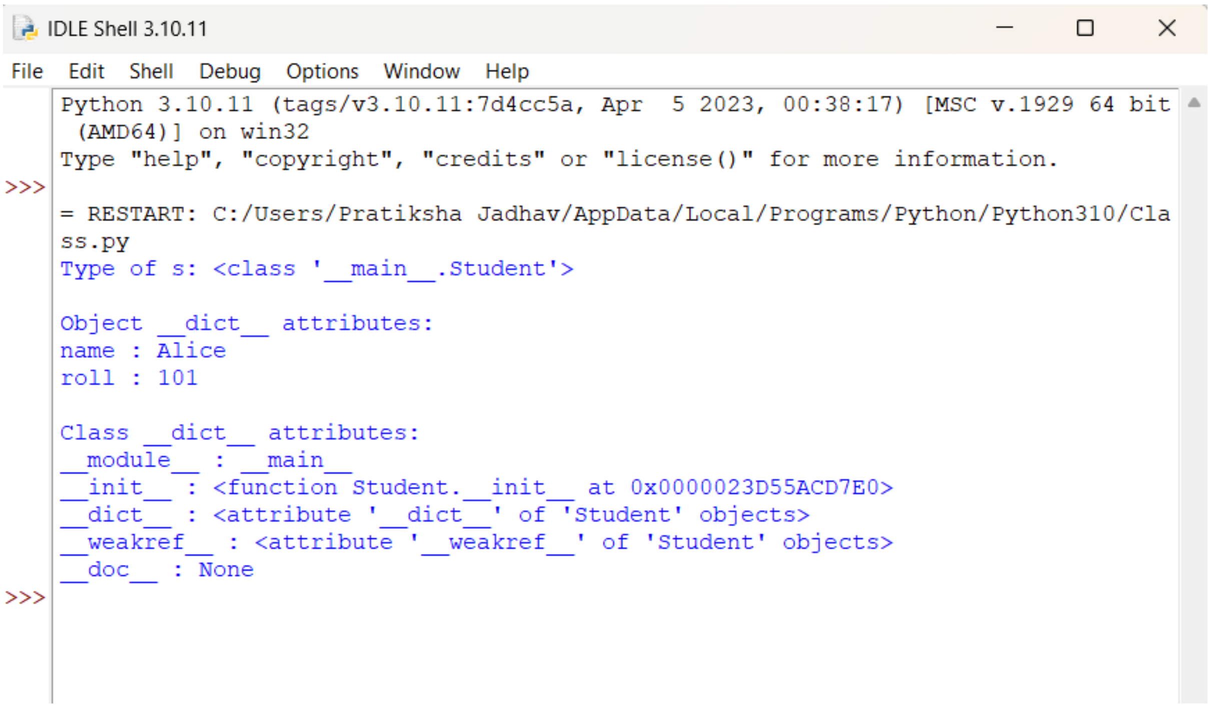


```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Pr
actical Assignment MCA/Age Exception.py
Enter your age: 21
Age is valid!
>>>
= RESTART: C:/Users/Pratiksha Jadhav/OneDrive/Desktop/PYTHON PROGRAMS LAB ASS/Pr
actical Assignment MCA/Age Exception.py
Enter your age: 17
Age Error: Age must be 18 or above
>>>
```

Assignment No : 3

Q1. Write a simple Python class named Student and display its type. Also display the dict attribute keys and values of the_module_attribute of the Student class.

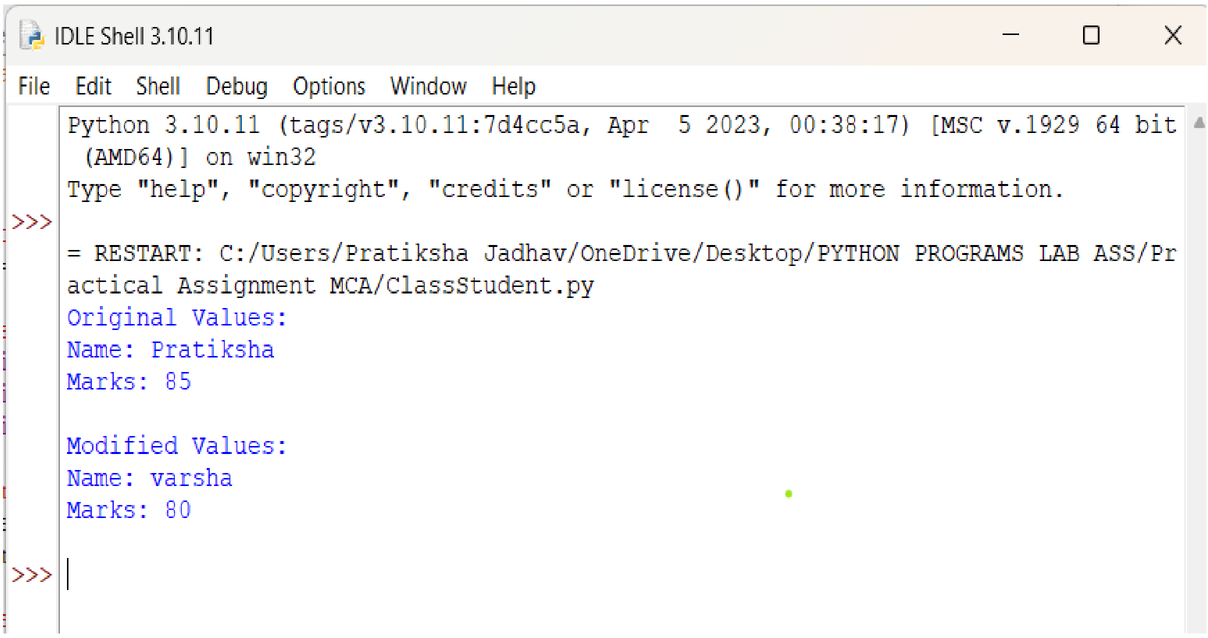
```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll
s = Student("Alice", 101)
print("Type of s:", type(s))
print("\nObject __dict__ attributes:")
for key, value in s.__dict__.items():
    print(key, ":", value)
# display the __dict__ attribute of the class
print("\nClass __dict__ attributes:")
for key, value in Student.__dict__.items():
    print(key, ":", value)
```



```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Cla
ss.py
Type of s: <class '__main__.Student'>

Object __dict__ attributes:
name : Alice
roll : 101

Class __dict__ attributes:
__module__ : __main__
__init__ : <function Student.__init__ at 0x0000023D55ACD7E0>
__dict__ : <attribute '__dict__' of 'Student' objects>
__weakref__ : <attribute '__weakref__' of 'Student' objects>
__doc__ : None
>>>
```

Q2. Write a Python class Student with two attributes, student_name and marks. Modify any attribute value of the class and show the original and modified values of attributes.

```
class Student:
    def __init__(self, student_name, marks):
        self.student_name = student_name
        self.marks = marks
# create object
s = Student("Pratiksha", 85)
# show original values
print("Original Values:")
print("Name:", s.student_name)
print("Marks:", s.marks)
# modify attributes
s.student_name = "varsha"
s.marks = 80
print("\nModified Values:")
print("Name:", s.student_name)
print("Marks:", s.marks)
```



Q3. Write a Python class to convert an integer to a Roman numeral.

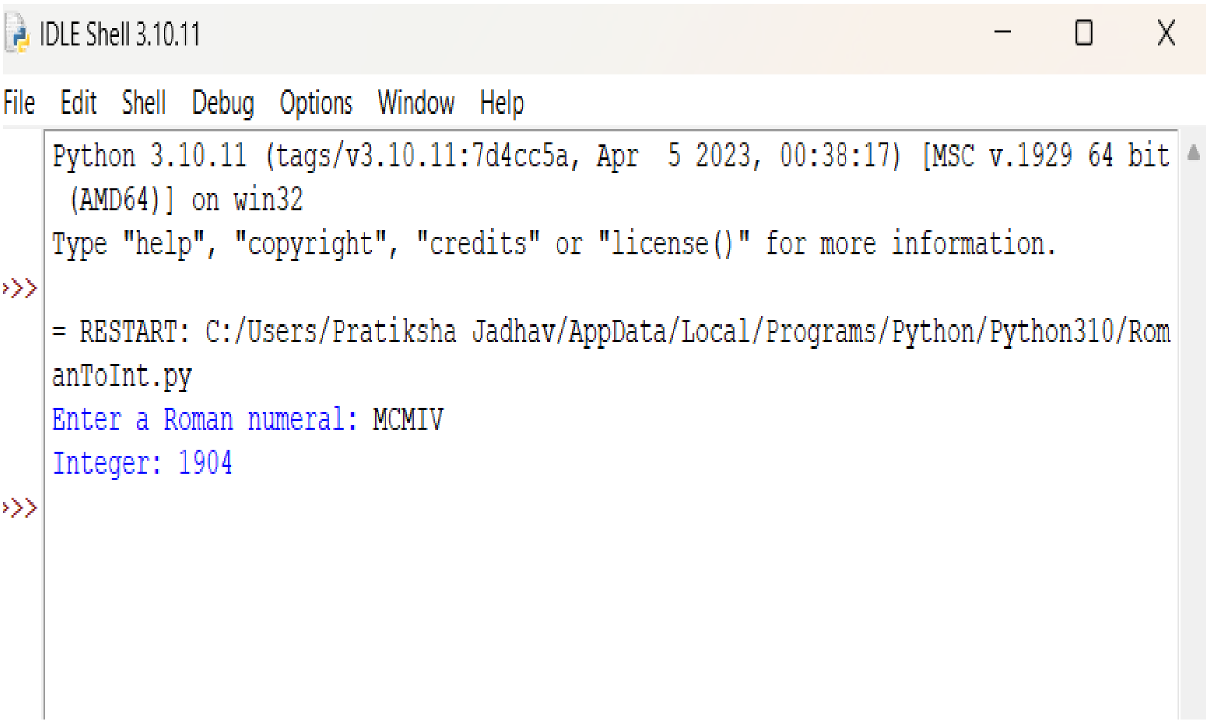
```
class IntegerToRoman:
    def __init__(self, number):
        self.number = number
    def convert(self):
        val = [
            1000, 900, 500, 400,
            100, 90, 50, 40,
            10, 9, 5, 4, 1
        ]
        syms = [
            "M", "CM", "D", "CD",
            "C", "XC", "L", "XL",
            "X", "IX", "V", "IV", "I"
        ]
        num = self.number
        roman = ""
        for i in range(len(val)):
            while num >= val[i]:
                roman += syms[i]
                num -= val[i]
        return roman
# Example usage
n = int(input("Enter an integer: "))
converter = IntegerToRoman(n)
print("Roman numeral:", converter.convert())
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Con
vertInttoRoman.py
Enter an integer: 5
Roman numeral: V
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Con
vertInttoRoman.py
Enter an integer: 11
Roman numeral: XI
>>>
```

Q4. Write a Python class to convert a Roman numeral to an integer.

```
class RomanToInteger:
    def __init__(self, roman):
        self.roman = roman\
    def convert(self):
        roman_values = {
            'I': 1, 'V': 5, 'X': 10, 'L': 50,
            'C': 100, 'D': 500, 'M': 1000
        }

        total = 0
        prev_value = 0
        for char in reversed(self.roman):
            value = roman_values[char]
            if value < prev_value:
                total -= value
```

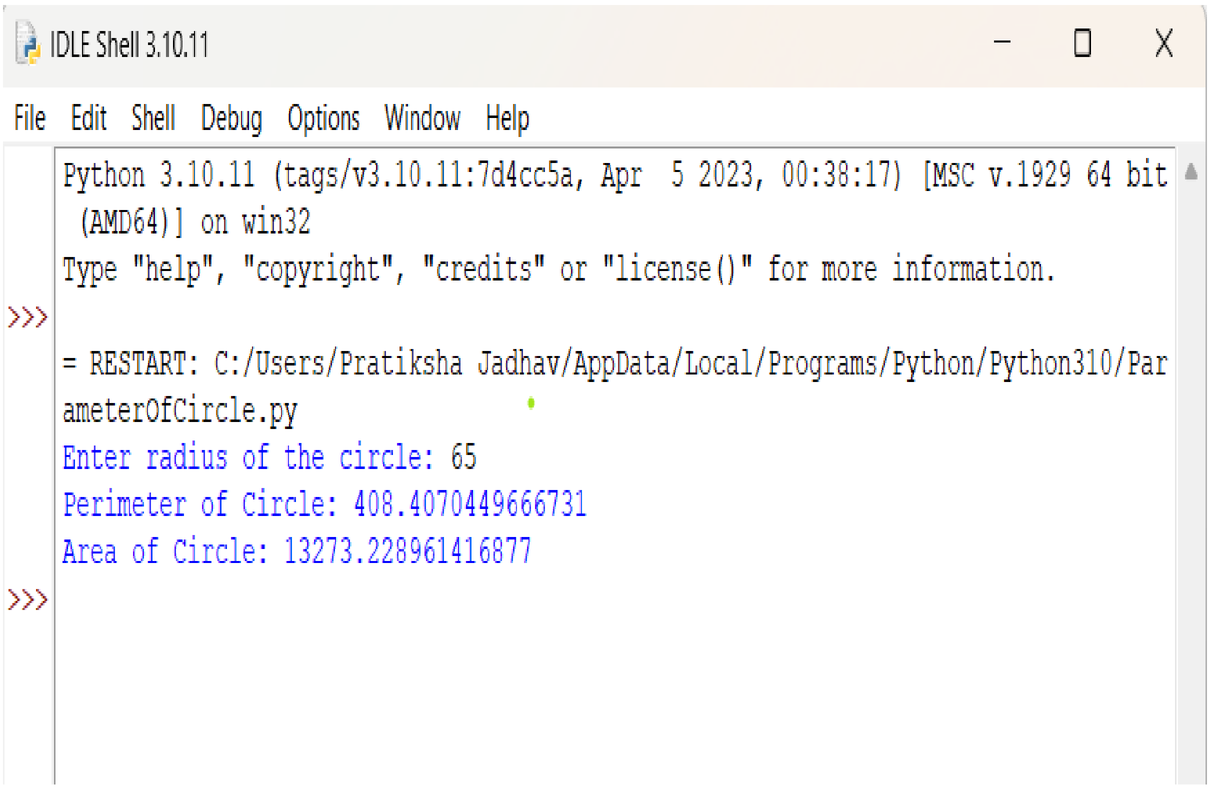
```
            else:
                total += value
            prev_value = value
        return total
roman = input("Enter a Roman numeral: ").upper()
converter = RomanToInteger(roman)
print("Integer:", converter.convert())
```



Q5. Write a program to create a class Shape with methods called get_perimeter and get area. Create a subclass called Circle that overrides the get_perimeter and get_area methods to calculate the area and perimeter of a circle.

```
import math
class Shape:
    def get_perimeter(self):
        return "Perimeter formula not defined"
```

```
    def get_area(self):
        return "Area formula not defined"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def get_perimeter(self):
        return 2 * math.pi * self.radius   # circumference

    def get_area(self):
        return math.pi * (self.radius ** 2)

# Example usage
r = float(input("Enter radius of the circle: "))
c = Circle(r)

print("Perimeter of Circle:", c.get_perimeter())
print("Area of Circle:", c.get_area())
```

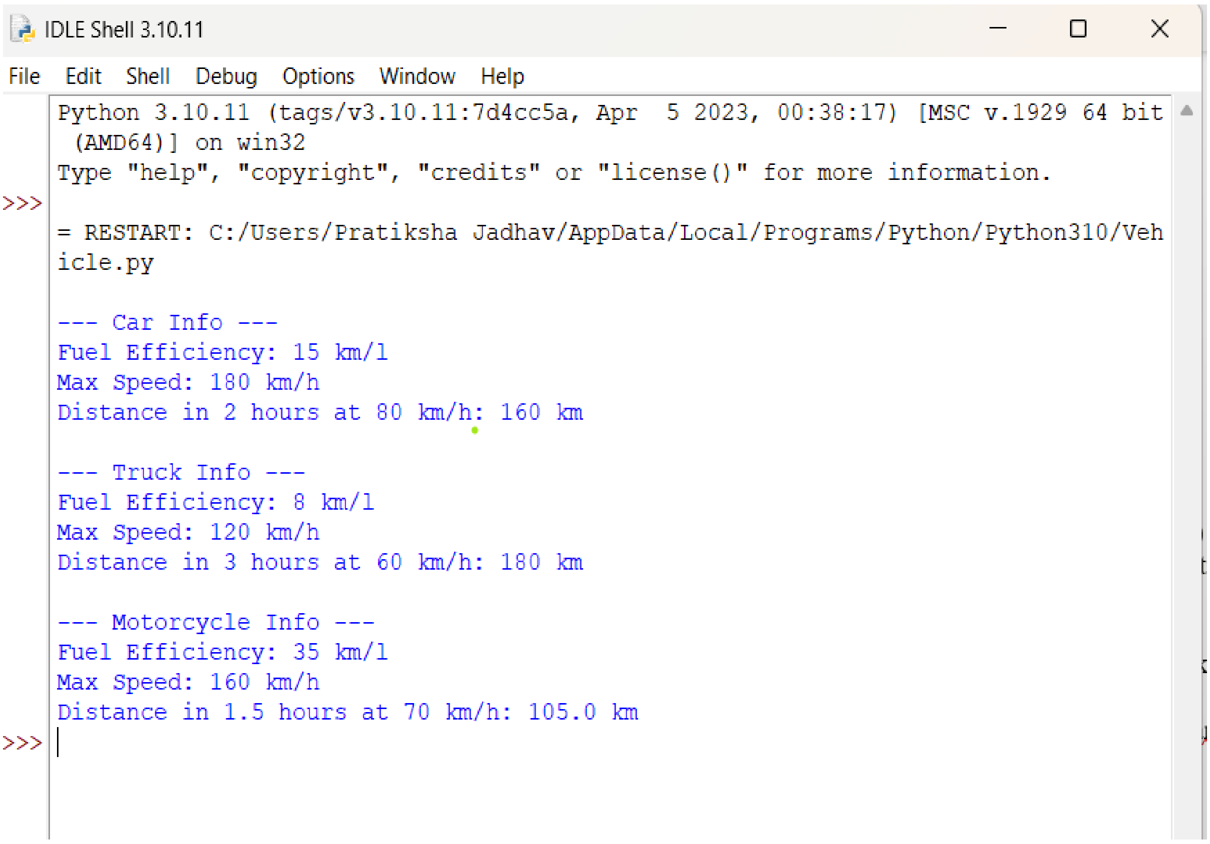


```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Par
ameterOfCircle.py
Enter radius of the circle: 65
Perimeter of Circle: 408.4070449666731
Area of Circle: 13273.228961416877
>>>
```

Q6. Write a program to create a Vehicle class hierarchy. The base class should be Vehicle, with subclasses Truck, Car, and Motorcycle. Each subclass should have properties such as make, model, year, and fuel_type. Implement methods for calculating fuel efficiency, distance traveled, and maximum speed.

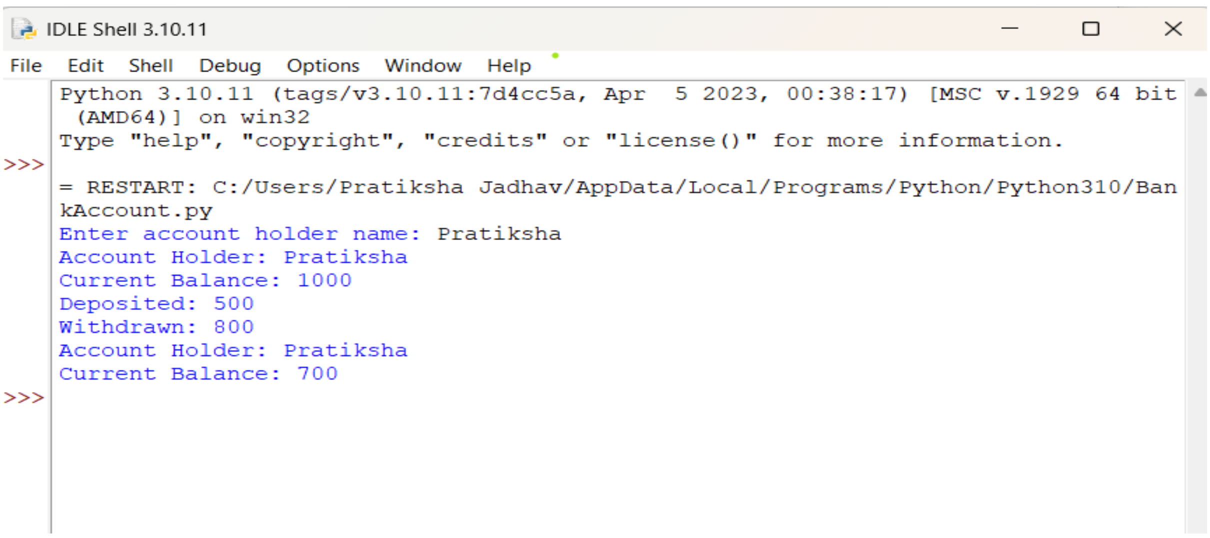
```
class Vehicle:
    def __init__(self, make, model, year, fuel_type):
        self.make = make
        self.model = model
        self.year = year
        self.fuel_type = fuel_type

    def fuel_efficiency(self):
        return "Fuel efficiency not defined"

    def distance_traveled(self, hours, speed):
        return hours * speed

    def max_speed(self):
        return "Max speed not defined"


class Car(Vehicle):
    def fuel_efficiency(self):
        return 15  # km per liter (example)

    def max_speed(self):
        return 180  # km/h


class Truck(Vehicle):
    def fuel_efficiency(self):
        return 8  # km per liter (example)

    def max_speed(self):
        return 120  # km/h


class Motorcycle(Vehicle):
    def fuel_efficiency(self):
        return 35  # km per liter (example)
```

```
    def max_speed(self):
        return 160  # km/h

# Example usage
car = Car("Toyota", "Corolla", 2020, "Petrol")
truck = Truck("Volvo", "FH16", 2019, "Diesel")
bike = Motorcycle("Yamaha", "R15", 2023, "Petrol")

print("\n--- Car Info ---")
print("Fuel Efficiency:", car.fuel_efficiency(), "km/l")
print("Max Speed:", car.max_speed(), "km/h")
print("Distance in 2 hours at 80 km/h:", car.distance_traveled(2, 80), "km")

print("\n--- Truck Info ---")
print("Fuel Efficiency:", truck.fuel_efficiency(), "km/l")
print("Max Speed:", truck.max_speed(), "km/h")
print("Distance in 3 hours at 60 km/h:", truck.distance_traveled(3, 60), "km")

print("\n--- Motorcycle Info ---")
print("Fuel Efficiency:", bike.fuel_efficiency(), "km/l")
print("Max Speed:", bike.max_speed(), "km/h")
print("Distance in 1.5 hours at 70 km/h:", bike.distance_traveled(1.5, 70), "km")
```



Q7. Implement a class BankAccount with methods for deposit, withdrawal, and displaying the account balance.

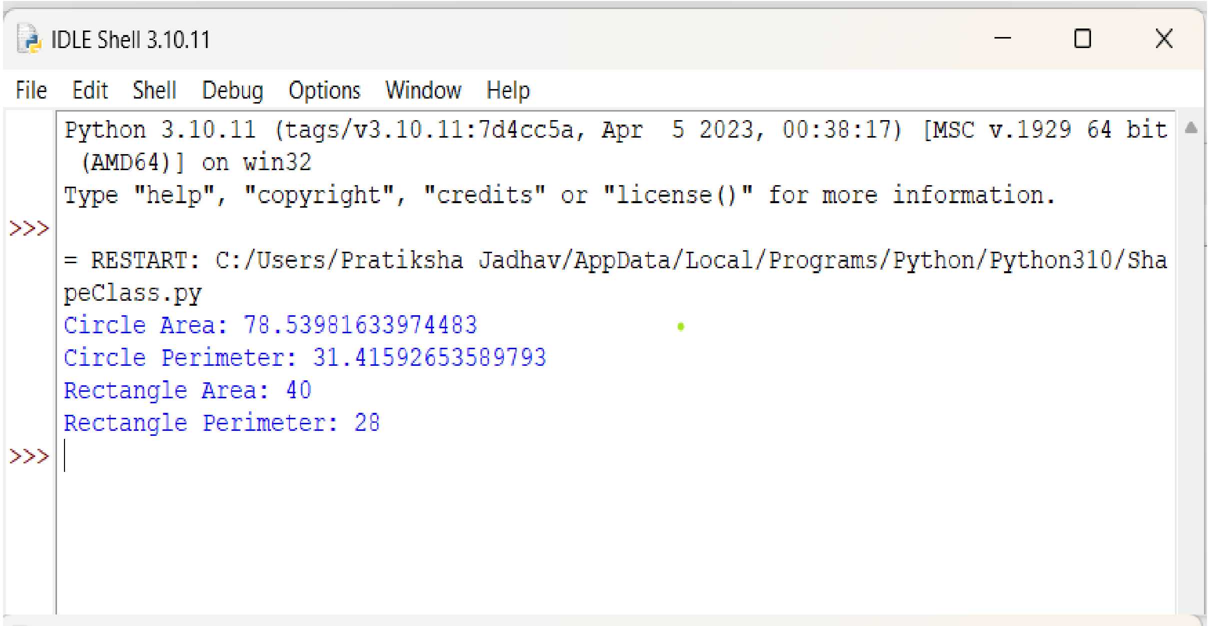
```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited: {amount}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance!")
        else:
            self.balance -= amount
            print(f"Withdrawn: {amount}")

    def display_balance(self):
        print(f"Account Holder: {self.account_holder}")
        print(f"Current Balance: {self.balance}")

name = input("Enter account holder name: ")
acc = BankAccount(name, 1000)

acc.display_balance()
acc.deposit(500)
acc.withdraw(800)
acc.display_balance()
```

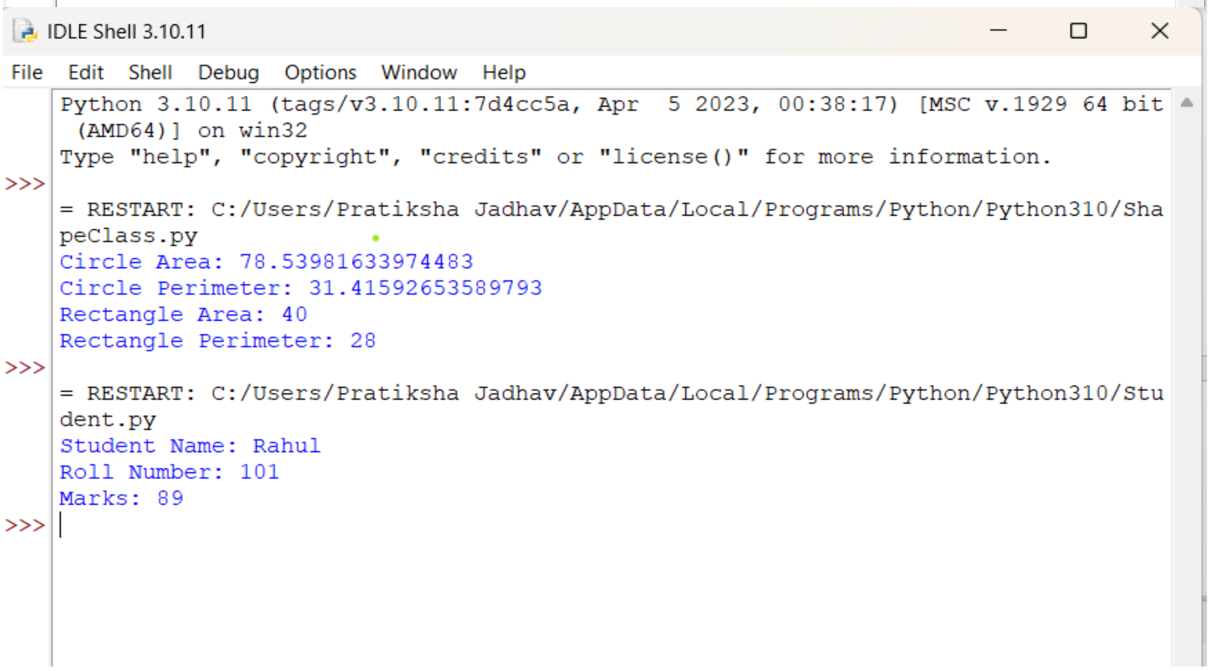


Q8. Develop a hierarchy of shapes (e.g., Shape, Circle, Rectangle) with area and perimeter methods.Student class.

```
import math

class Shape:
    def area(self):
        return "Area not defined"

    def perimeter(self):
        return "Perimeter not defined"


class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.radius


class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)
# Example usage
c = Circle(5)
r = Rectangle(10, 4)

print("Circle Area:", c.area())
print("Circle Perimeter:", c.perimeter())
print("Rectangle Area:", r.area())
print("Rectangle Perimeter:", r.perimeter())
```

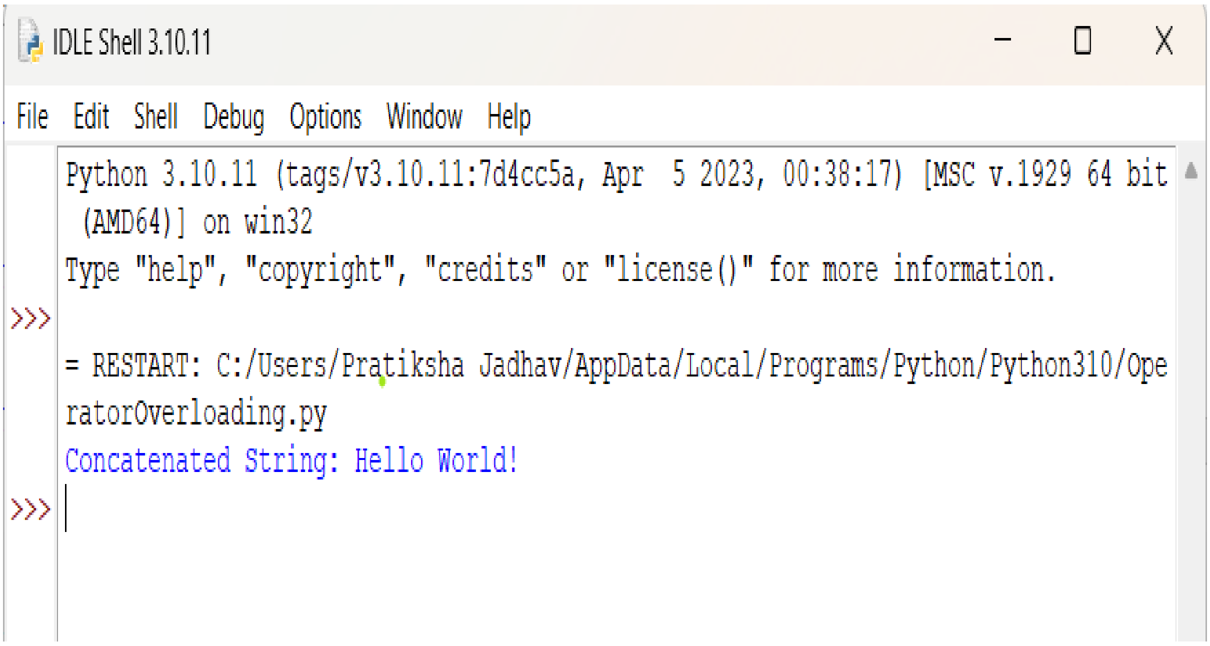
```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks
    def display(self):
        print("Student Name:", self.name)
        print("Roll Number:", self.roll_no)
        print("Marks:", self.marks)
s = Student("Rahul", 101, 89)
s.display()
```





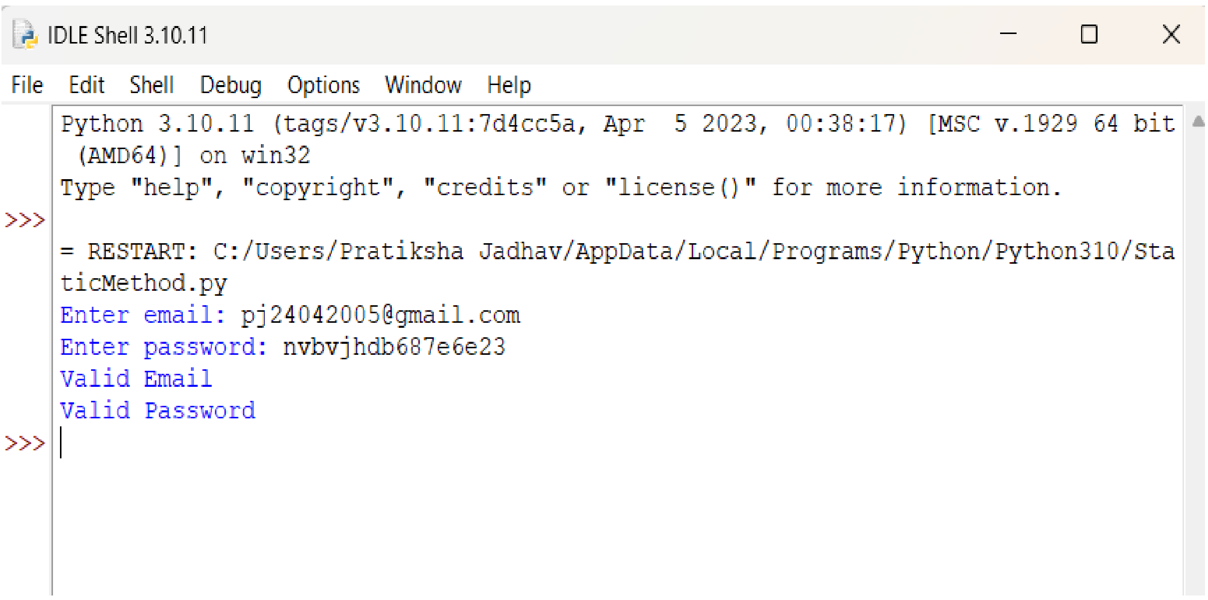
Q9. Overload the + operator for concatenating two custom string objects

```
class CustomString:
    def __init__(self, text):
        self.text = text

    def __add__(self, other):
        return CustomString(self.text + other.text)

    def __str__(self):
        return self.text

# Example usage
s1 = CustomString("Hello ")
s2 = CustomString("World!")

s3 = s1 + s2   # Using overloaded + operator

print("Concatenated String:", s3)
```



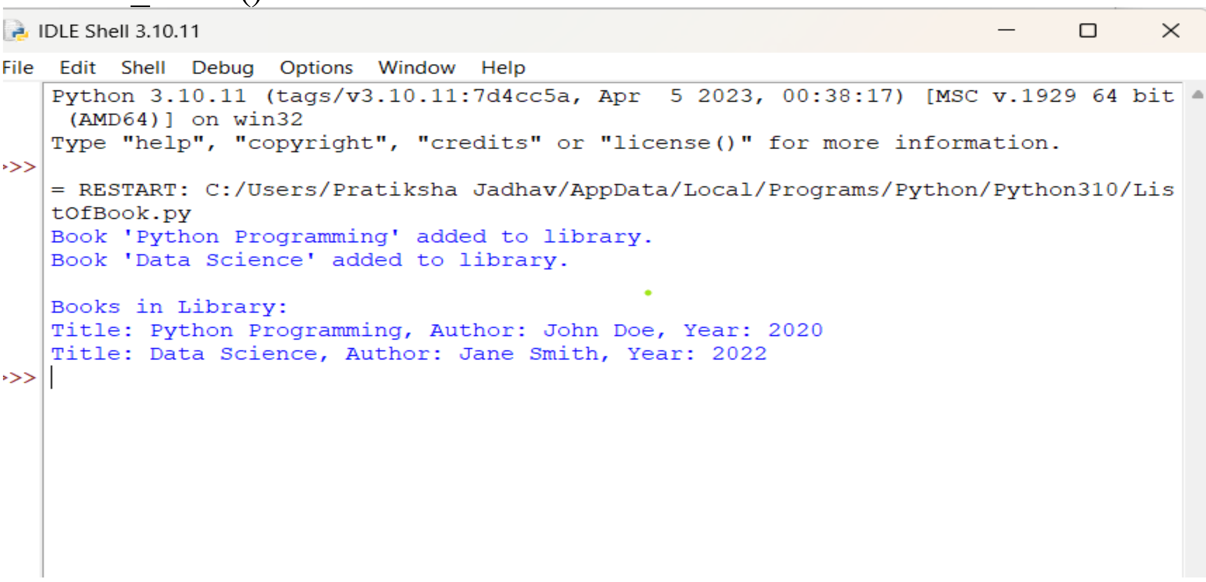
Q10. Create a static method to validate user credentials (e.g., email format, password length).

```
import re
class Validator:
    @staticmethod
    def validate_email(email):
        pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
        return re.match(pattern, email) is not None

    @staticmethod
    def validate_password(password):
        return len(password) >= 8

email = input("Enter email: ")
password = input("Enter password: ")

if Validator.validate_email(email):
    print("Valid Email")
else:
    print("Invalid Email")

if Validator.validate_password(password):
    print("Valid Password")
else:
    print("Password must be at least 8 characters long")
```



Q 11. Implement a class Library containing a list of Book objects

```
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year
    def display(self):
        print(f"Title: {self.title}, Author: {self.author}, Year: {self.year}")
class Library:
    def __init__(self):
        self.books = []     # list of Book objects
    def add_book(self, book):
        self.books.append(book)
        print(f"Book '{book.title}' added to library.")

    def show_books(self):
        if not self.books:
            print("No books in library.")
        else:
            print("\nBooks in Library:")
            for book in self.books:
                book.display()
b1 = Book("Python Programming", "John Doe", 2020)
b2 = Book("Data Science", "Jane Smith", 2022)

lib = Library()
lib.add_book(b1)
lib.add_book(b2)
lib.show_books()
```

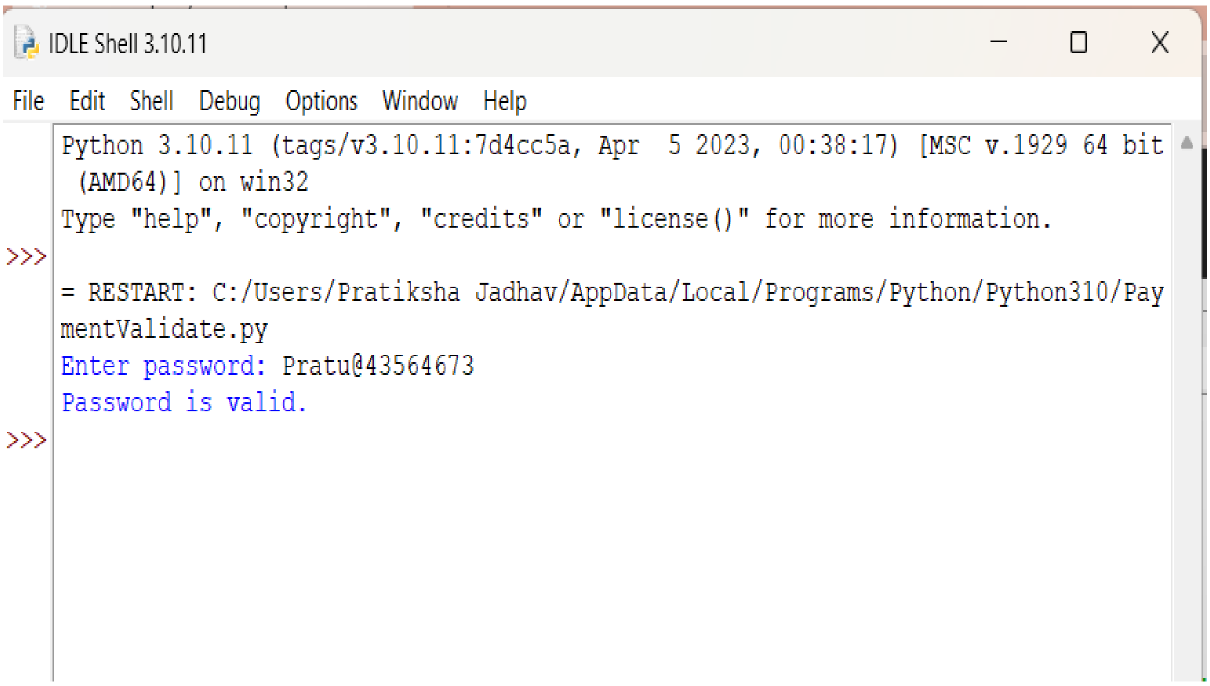


Q12. Demonstrate method overriding in a superclass-subclass relationship for a payment system (e.g., Payment, CardPayment).

```
class Payment:
    def process_payment(self, amount):
        print(f"Processing a generic payment of ₹{amount}")

class CardPayment(Payment):
    def process_payment(self, amount):
        print(f"Processing card payment of ₹{amount}")
        print("Payment successful using Debit/Credit Card")

p = Payment()
cp = CardPayment()

print("---- Payment ----")
p.process_payment(500)

print("\n---- Card Payment ----")
cp.process_payment(500)
```

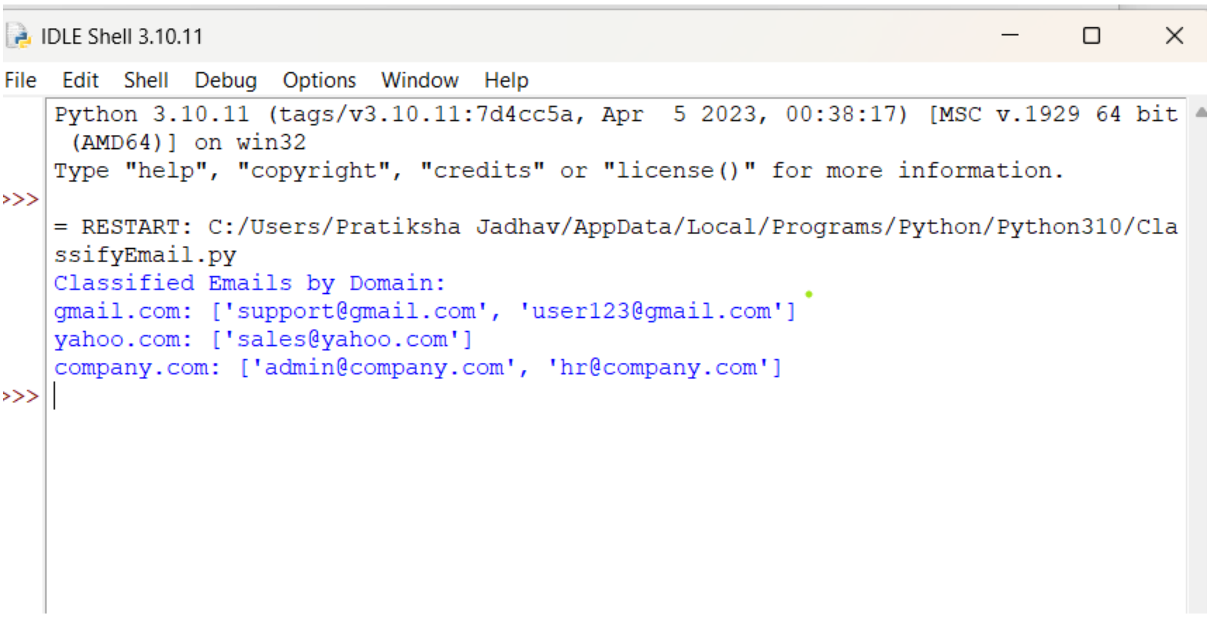
Q 13. Write a program to validate a password that meets certain criteria (length, uppercase, digit, special character).

```
import re

def validate_password(password):
    if len(password) < 8:
        return "Password must be at least 8 characters long."

    if not re.search(r"[A-Z]", password):
        return "Password must contain at least one uppercase letter."

    if not re.search(r"[0-9]", password):
        return "Password must contain at least one digit."

    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        return "Password must contain at least one special character."

    return "Password is valid."

password = input("Enter password: ")
print(validate_password(password))
```



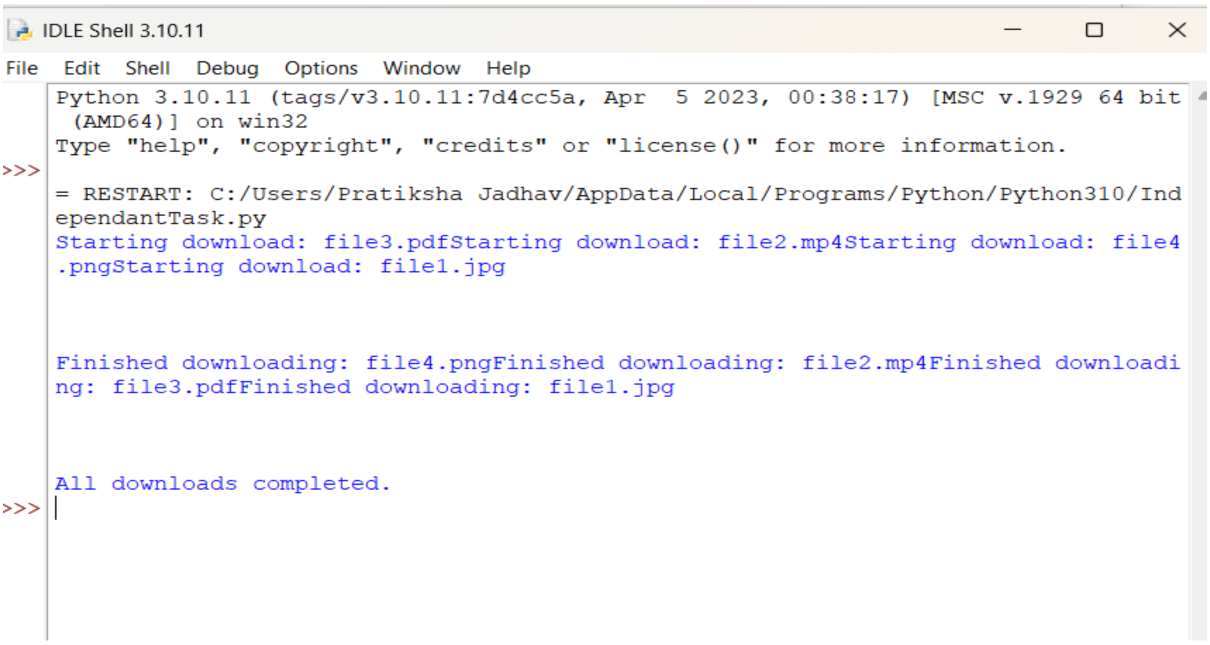
Q 14. Parse email addresses from a given text and classify them by domain.

```
import re

def classify_emails(text):
    pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
    emails = re.findall(pattern, text)
    domain_dict = {}

    for email in emails:
        domain = email.split("@")[1]
        if domain not in domain_dict:
            domain_dict[domain] = []
        domain_dict[domain].append(email)

    return domain_dict
text = """
Contact us at support@gmail.com or sales@yahoo.com.
You can also reach admin@company.com and hr@company.com
for further queries. Backup: user123@gmail.com
"""
result = classify_emails(text)

print("Classified Emails by Domain:")
for domain, email_list in result.items():
    print(f"{domain}: {email_list}")
```



Q15. Create threads to perform independent tasks such as downloading multiple files Simultaneously

```
import threading
import time

def download_file(filename):
    print(f"Starting download: {filename}")
    time.sleep(2)   # Simulate download time
    print(f"Finished downloading: {filename}")

# List of files
files = ["file1.jpg", "file2.mp4", "file3.pdf", "file4.png"]

threads = []

# Create and start threads
for file in files:
    t = threading.Thread(target=download_file, args=(file,))
    threads.append(t)
    t.start()

# Wait for all threads to finish
for t in threads:
    t.join()
print("All downloads completed.")
```



```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Ind
ependantTask.py
Starting download: file3.pdfStarting download: file2.mp4Starting download: file4
.pngStarting download: file1.jpg


Finished downloading: file4.pngFinished downloading: file2.mp4Finished downloadi
ng: file3.pdfFinished downloading: file1.jpg


All downloads completed.
>>>
```

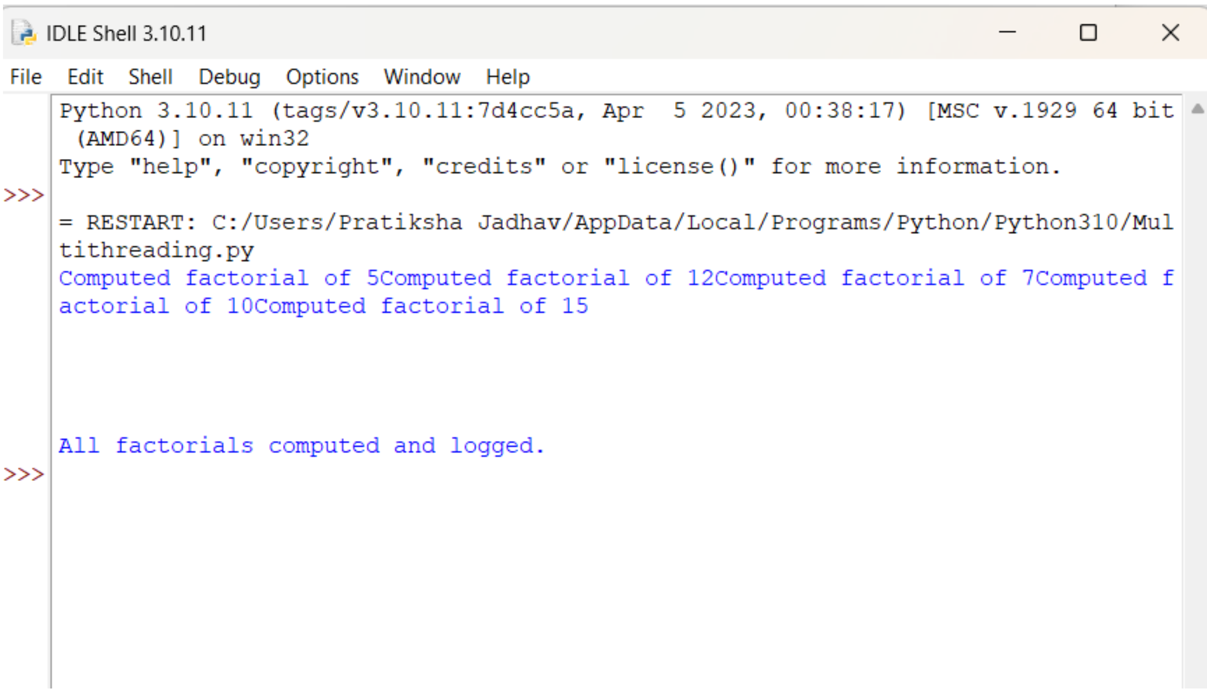
Q 16. Write a Python program to create two threads. The first thread will print even numbers, and the second thread will print odd numbers. Demonstrate the concept of multithreading.

```
import threading
import time
def print_even():
    for i in range(0, 21, 2):
        print("Even:", i)
        time.sleep(0.2)
def print_odd():
    for i in range(1, 21, 2):
        print("Odd :", i)
        time.sleep(0.2)

t1 = threading.Thread(target=print_even)
t2 = threading.Thread(target=print_odd)
t1.start()
t2.start()
t1.join()
t2.join()
print("Both threads finished execution.")
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Pratiksha Jadhav/AppData/Local/Programs/Python/Python310/Two
Thread.py
Even:Odd : 0
1
Even:Odd :  23

Odd :Even:  54

Odd :Even:  76

Odd :Even:  98

Odd :Even:  1110

Even:Odd :  1213

Even:Odd :  1415

Odd :Even:  1716

Odd :Even:  1918

Even: 20
Both threads finished execution.
>>>
```

Q17. Use multithreading to compute factorials of multiple numbers and log the results into a file.

```
import threading
import math

def compute_factorial(n):
    result = math.factorial(n)
    with open("factorial_log.txt", "a") as file:
        file.write(f"Factorial of {n} = {result}\n")
    print(f"Computed factorial of {n}")

numbers = [5, 7, 10, 12, 15]

threads = []

# Create and start threads
for num in numbers:
    t = threading.Thread(target=compute_factorial, args=(num,))
    threads.append(t)
    t.start()
for t in threads:
    t.join()
print("All factorials computed and logged.")
```



Assignment No : 4

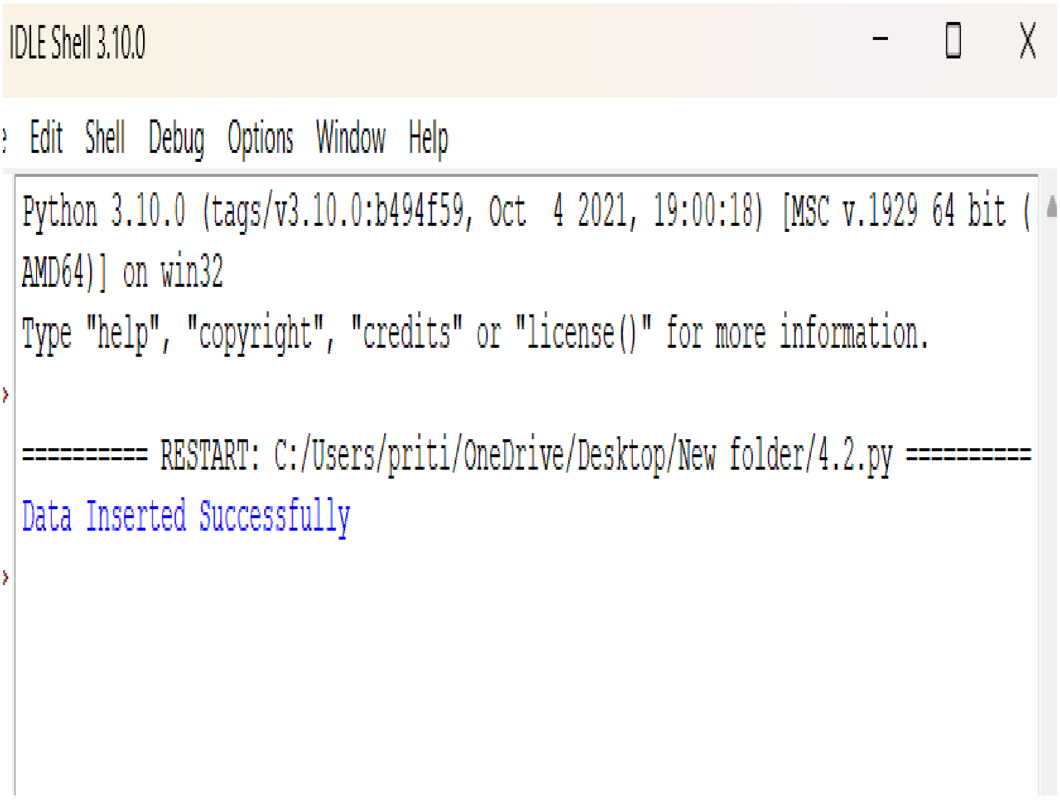
Q1. Develop a Python program to create, read, update, and delete documents in a MongoDB collection.

```
from pymongo import MongoClient
# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")
db = client["assignment4"]
collection = db["demo_collection"]
# ---- CREATE ----
document = {"name": "Amit", "age": 25, "city": "Pune"}
collection.insert_one(document)
print("Document Inserted")
# ---- READ ----
print("\nAll Documents:")
for doc in collection.find():
    print(doc)
# ---- UPDATE ----
collection.update_one({"name": "Amit"}, {"$set": {"age": 30}})
print("\nAfter Update:")
print(collection.find_one({"name": "Amit"}))
# ---- DELETE ----
collection.delete_one({"name": "Amit"})
print("\nAfter Delete:")
for doc in collection.find():
    print(doc)
```

```
DLE Shell 3.10.0
 Edit  Shell  Debug  Options  Window  Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win
Type "help", "copyright", "credits" or "license()" for more information.

========== RESTART: C:/Users/priti/OneDrive/Desktop/New folder/4.1.py ==========
Document Inserted

All Documents:
{'_id': ObjectId('6932c5e04b6890028c991d5d'), 'name': 'Amit', 'age': 25, 'city': 'Pune'}

After Update:
{'_id': ObjectId('6932c5e04b6890028c991d5d'), 'name': 'Amit', 'age': 30, 'city': 'Pune'}

After Delete:
```

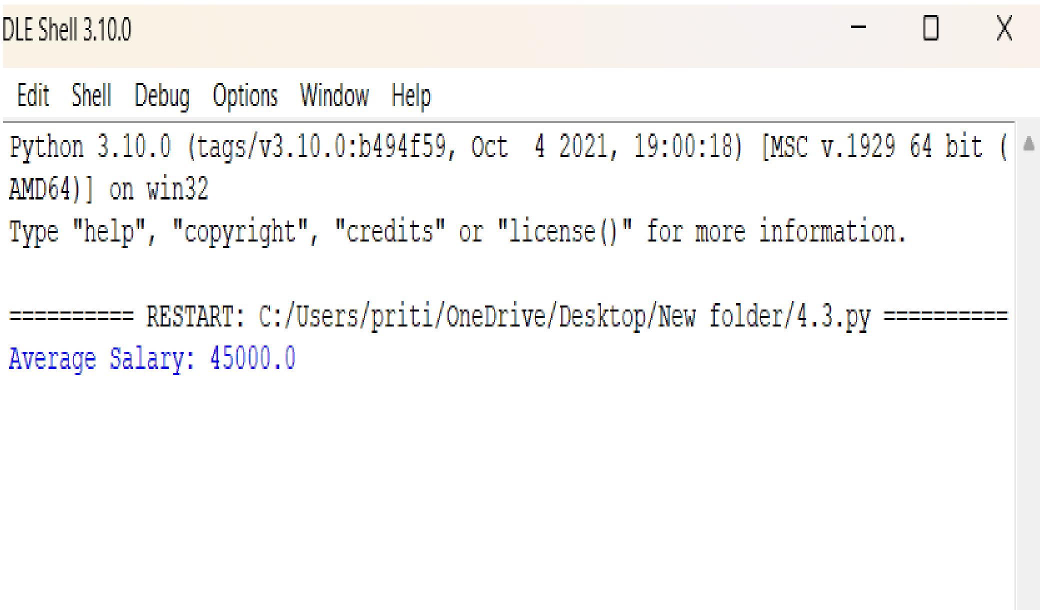
Q2. Write a program to insert valid data into MongoDB using exception handling for errors.

```
from pymongo import MongoClient, errors
client = MongoClient("mongodb://localhost:27017/")
db = client["assignment4"]
collection = db["student_records"]
data = {"name": "Riya", "age": 20}

try:
    if not isinstance(data["age"], int):
        raise ValueError("Age must be a number")

    collection.insert_one(data)
    print("Data Inserted Successfully")
except ValueError as ve:
    print("Validation Error:", ve)
except errors.PyMongoError as pe:
    print("MongoDB Error:",pe)
```



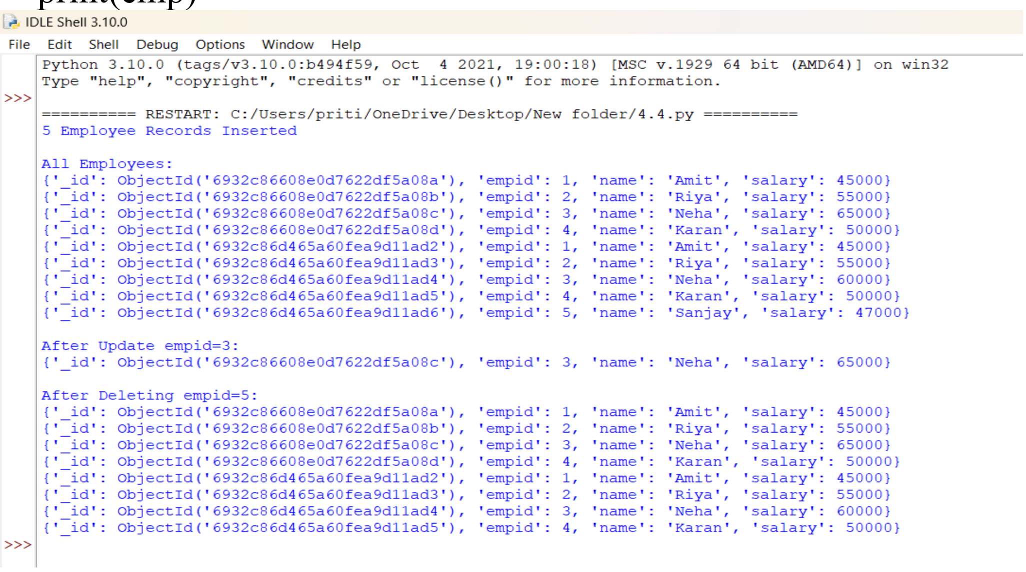
Q3. Use MongoDB's aggregation pipeline to find the average salary of employees in a collection.

```
client = MongoClient("mongodb://localhost:27017/")
db = client["assignment4"]
collection = db["employees"]
# Sample Data (Run once only)
sample = [
    {"name": "John", "salary": 40000},
    {"name": "Sara", "salary": 50000},
    {"name": "Amit", "salary": 45000},
]
collection.insert_many(sample)

pipeline = [
    {"$group": {"_id": None, "avg_salary": {"$avg": "$salary"}}}
]
result = list(collection.aggregate(pipeline))
print("Average Salary:", result[0]["avg_salary"])
```



Q4. Develop a Python program to create a database name employee. With the collection name empinfo. Add 5 douments in the collection. Perform read, update, and delete operations on documents in a MongoDB..

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["employee"]
collection = db["empinfo"]
# Add 5 documents
employees = [
    {"empid": 1, "name": "Amit", "salary": 45000},
    {"empid": 2, "name": "Riya", "salary": 55000},
    {"empid": 3, "name": "Neha", "salary": 60000},
    {"empid": 4, "name": "Karan", "salary": 50000},
    {"empid": 5, "name": "Sanjay", "salary": 47000}
]
collection.insert_many(employees)
print("5 Employee Records Inserted")
# ---- READ ----
print("\nAll Employees:")
for emp in collection.find():
    print(emp)
# ---- UPDATE ----
collection.update_one({"empid": 3}, {"$set": {"salary": 65000}})
print("\nAfter Update empid=3:")
print(collection.find_one({"empid": 3}))
# ---- DELETE ----
collection.delete_one({"empid": 5})
print("\nAfter Deleting empid=5:")
for emp in collection.find():
    print(emp)
```



```
IDLE Shell 3.10.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/priti/OneDrive/Desktop/New folder/4.4.py ==========
5 Employee Records Inserted

All Employees:
{'_id': ObjectId('6932c86608e0d7622df5a08a'), 'empid': 1, 'name': 'Amit', 'salary': 45000}
{'_id': ObjectId('6932c86608e0d7622df5a08b'), 'empid': 2, 'name': 'Riya', 'salary': 55000}
{'_id': ObjectId('6932c86608e0d7622df5a08c'), 'empid': 3, 'name': 'Neha', 'salary': 65000}
{'_id': ObjectId('6932c86608e0d7622df5a08d'), 'empid': 4, 'name': 'Karan', 'salary': 50000}
{'_id': ObjectId('6932c86d465a60fea9d11ad2'), 'empid': 1, 'name': 'Amit', 'salary': 45000}
{'_id': ObjectId('6932c86d465a60fea9d11ad3'), 'empid': 2, 'name': 'Riya', 'salary': 55000}
{'_id': ObjectId('6932c86d465a60fea9d11ad4'), 'empid': 3, 'name': 'Neha', 'salary': 60000}
{'_id': ObjectId('6932c86d465a60fea9d11ad5'), 'empid': 4, 'name': 'Karan', 'salary': 50000}
{'_id': ObjectId('6932c86d465a60fea9d11ad6'), 'empid': 5, 'name': 'Sanjay', 'salary': 47000}

After Update empid=3:
{'_id': ObjectId('6932c86608e0d7622df5a08c'), 'empid': 3, 'name': 'Neha', 'salary': 65000}

After Deleting empid=5:
{'_id': ObjectId('6932c86608e0d7622df5a08a'), 'empid': 1, 'name': 'Amit', 'salary': 45000}
{'_id': ObjectId('6932c86608e0d7622df5a08b'), 'empid': 2, 'name': 'Riya', 'salary': 55000}
{'_id': ObjectId('6932c86608e0d7622df5a08c'), 'empid': 3, 'name': 'Neha', 'salary': 65000}
{'_id': ObjectId('6932c86608e0d7622df5a08d'), 'empid': 4, 'name': 'Karan', 'salary': 50000}
{'_id': ObjectId('6932c86d465a60fea9d11ad2'), 'empid': 1, 'name': 'Amit', 'salary': 45000}
{'_id': ObjectId('6932c86d465a60fea9d11ad3'), 'empid': 2, 'name': 'Riya', 'salary': 55000}
{'_id': ObjectId('6932c86d465a60fea9d11ad4'), 'empid': 3, 'name': 'Neha', 'salary': 60000}
{'_id': ObjectId('6932c86d465a60fea9d11ad5'), 'empid': 4, 'name': 'Karan', 'salary': 50000}
>>>
```

Assignment No : 5

Q1. Create a Django project with models for Product and Category, along with views to display them on a webpage.

PROGRAM:

1️⃣ Create Django Project

```
django-admin startproject myproject
cd myproject
```

2️⃣ Create App

```
python manage.py startapp shop
```

3️⃣ Add App in settings.py

Open myproject/settings.py → add:

```
INSTALLED_APPS = [
    ...
    'shop',
]
```

4️⃣ Create Models

Open shop/models.py:

```
class Category(models.Model):
    name = models.CharField(max_length=100)


class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    price = models.FloatField()
    description = models.TextField()
```

5️⃣ Make Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

6️⃣ Create View

Open shop/views.py:

```
def product_list(request):
    products = Product.objects.all()
    return render(request, 'product_list.html', {'products': products})
```

7️⃣ Create Template

Create file:

shop/templates/product_list.html

Add:

```
<h1>Product List</h1>
{% for p in products %}
  <p>{{ p.name }} - ₹{{ p.price }} - {{ p.category.name }}</p>
{% endfor %}
```

8️⃣ Create shop/urls.py

```
from django.urls import path
from .views import product_list

urlpatterns = [
    path('', product_list),
]
```

9️⃣ Add URL in project urls.py

Open myproject/urls.py:

```
from django.urls import path, include

urlpatterns = [
    path('', include('shop.urls')),
]
```

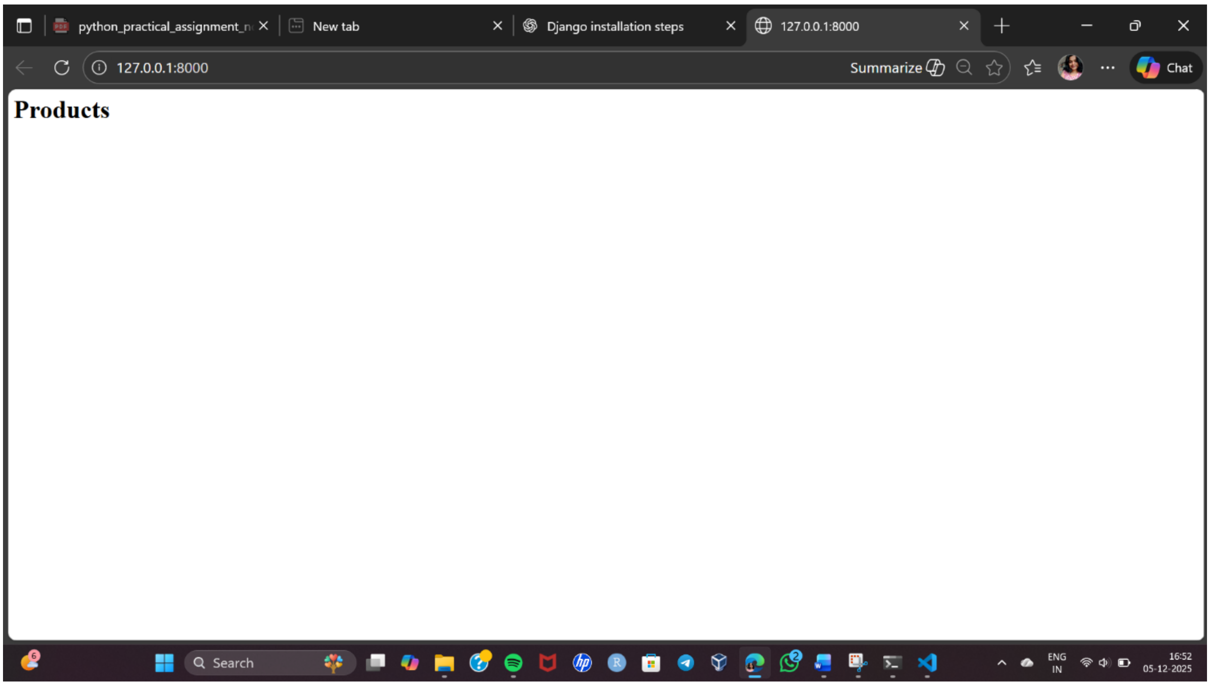
🔟 Run Server

python manage.py runserver

Open in browser:

http://127.0.0.1:8000/

You will see the Product List Page.



Q2. Implement user login, logout, and registration functionality in Django.

PROGRAM:

Step 1: Create Django Project & App

```
"C:\Program Files\Python310\python.exe" -m django startproject myproject
cd authproject
python manage.py startapp accounts
```

Add the app in settings.py

```
INSTALLED_APPS = [
    ...
    'accounts',
]
```

Step 2: Create Registration Form

Use Django’s built-in UserCreationForm.

accounts/forms.py

```
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
class RegisterForm(UserCreationForm):
    class Meta:
        model = User
        fields = ['username', 'password1', 'password2']
```

Step 3: Create Views for Register, Login & Logout

accounts/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.forms import AuthenticationForm
from .forms import RegisterForm
# Register User
def register_user(request):
```

```
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('login_user')
    else:
        form = RegisterForm()
    return render(request, 'register.html', {'form': form})
# Login User
def login_user(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user:
                login(request, user)
                return redirect('home')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})
# Logout User
def logout_user(request):
    logout(request)
    return redirect('login_user')
```

Step 4: Create URLs

authproject/urls.py

```
from django.contrib import admin
```

```python
from django.urls import path
from accounts import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.login_user, name='home'),  # default page
    path('register/', views.register_user, name='register_user'),
    path('login/', views.login_user, name='login_user'),
    path('logout/', views.logout_user, name='logout_user'),
]
```

Step 5: Create Templates

Create folder:

📁 accounts/templates/

Inside create:

- register.html
- login.html

register.html

```html
<h2>Register</h2>
<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Register</button>
</form>
<a href="{% url 'login_user' %}">Already have an account? Login</a>
```

login.html

```html
<h2>Login</h2>
<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Login</button>
```

</form>

Create new account

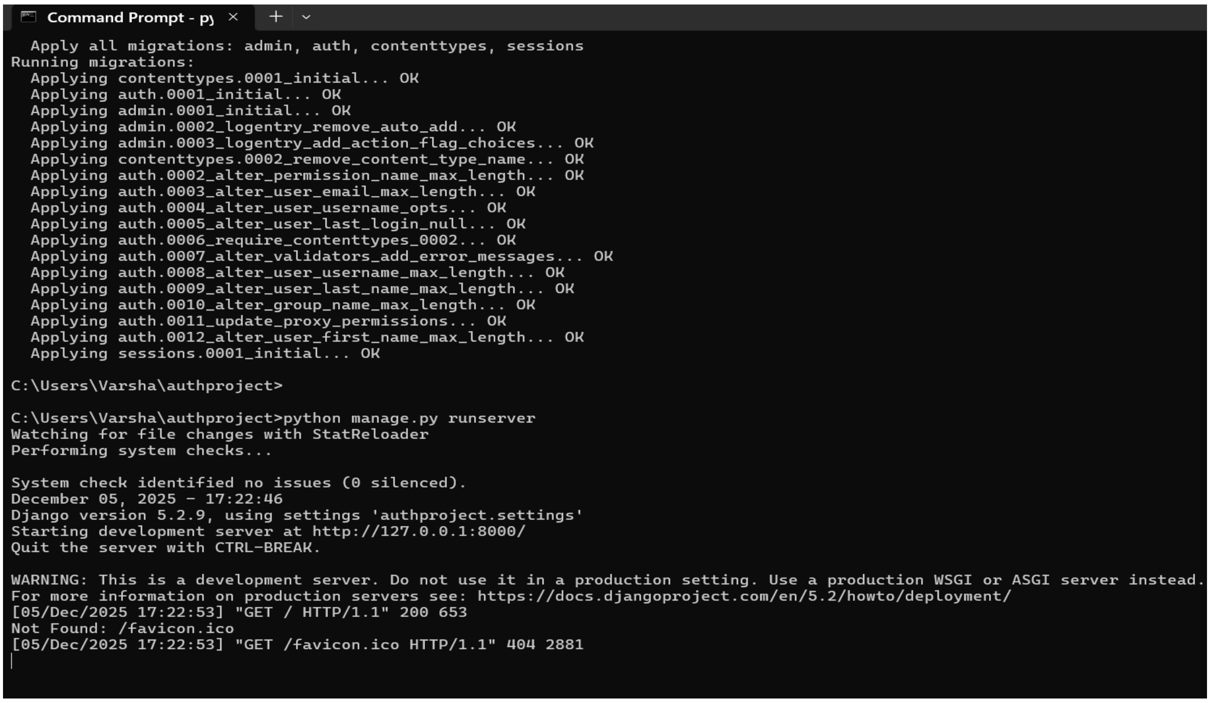
Step 6: Run Server

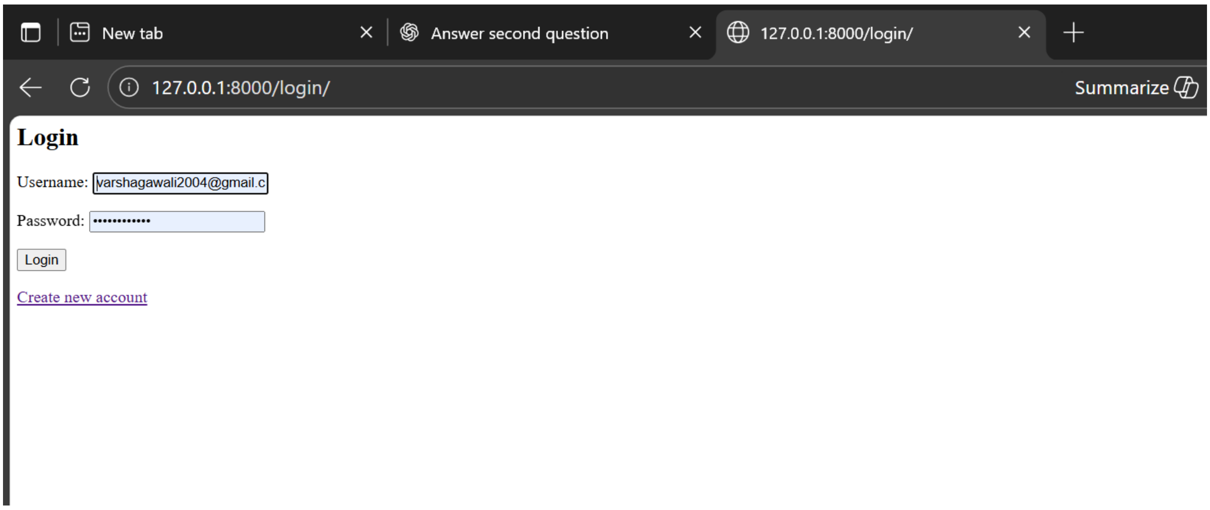
python manage.py migrate

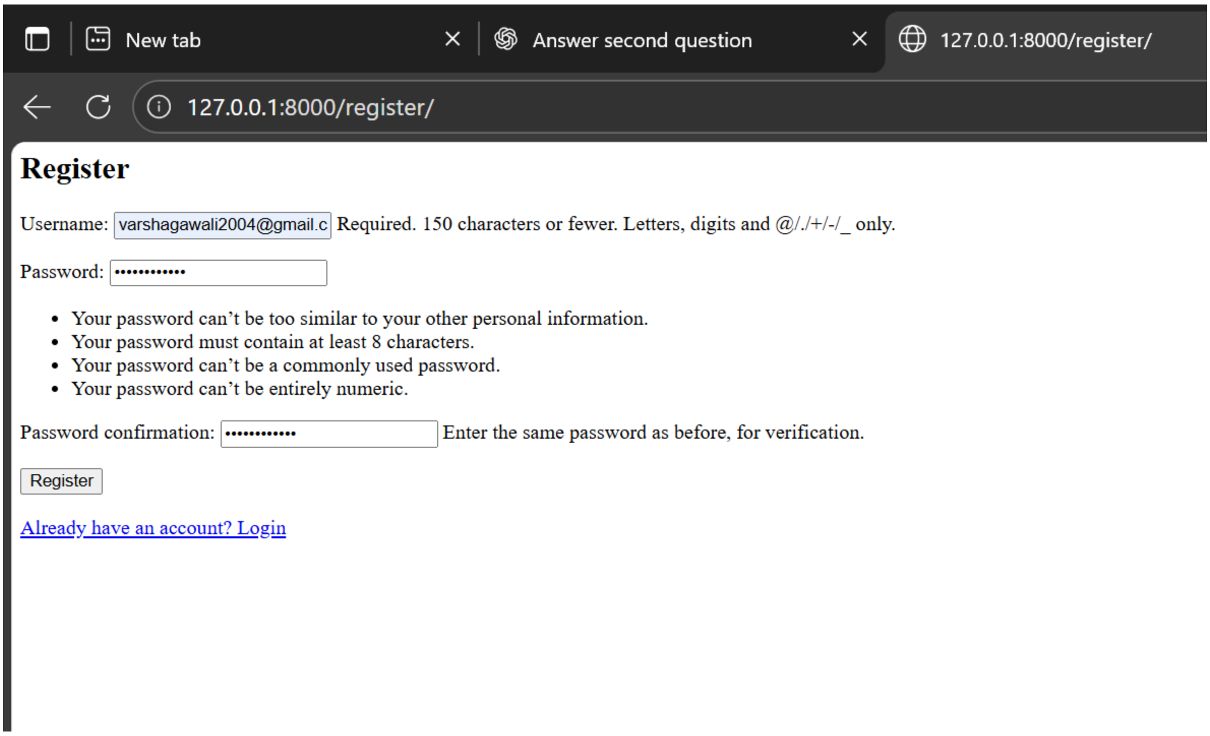
python manage.py runserver

Open in browser:

http://127.0.0.1:8000/






New tab
Answer second question
127.0.0.1:8000/register/
127.0.0.1:8000/register/
Register
Username: varshagawali2004@gmail.c Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.
Password: ••••••••••••
Your password can't be too similar to your other personal information.
Your password must contain at least 8 characters.
Your password can't be a commonly used password.
Your password can't be entirely numeric.
Password confirmation: •••••••••••• Enter the same password as before, for verification.
Register
Already have an account? Login

Q3. Build a REST API for a task management system where users can create, update, and delete tasks.

PROGRAM:

1. Install Django and DRF

```
pip install django djangorestframework
```

2. Create Django project and app

```
django-admin startproject taskmanager
cd taskmanager
python manage.py startapp tasks
```

3. Add apps to settings.py

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'tasks',
]
```

4. Create Task model (tasks/models.py)

```
from django.db import models
from django.contrib.auth.models import User
class Task(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    completed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
def __str__(self):
        return self.title
```

5. Make migrations

```
python manage.py makemigrations
python manage.py migrate
```

6. Create a Serializer (tasks/serializers.py)

```
from rest_framework import serializers
from .models import Task
class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = '__all__'
```

7. Create API Views (tasks/views.py)

```
from rest_framework import viewsets, permissions
from .models import Task
from .serializers import TaskSerializer
class TaskViewSet(viewsets.ModelViewSet):
    queryset = Task.objects.all()
    serializer_class = TaskSerializer
    permission_classes = [permissions.IsAuthenticated]
    def get_queryset(self):
        return self.queryset.filter(user=self.request.user)
    def perform_create(self, serializer):
        serializer.save(user=self.request.user)
```

8. Set up URLs (tasks/urls.py)

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import TaskViewSet
router = DefaultRouter()
router.register(r'tasks', TaskViewSet, basename='task')
urlpatterns = [
```

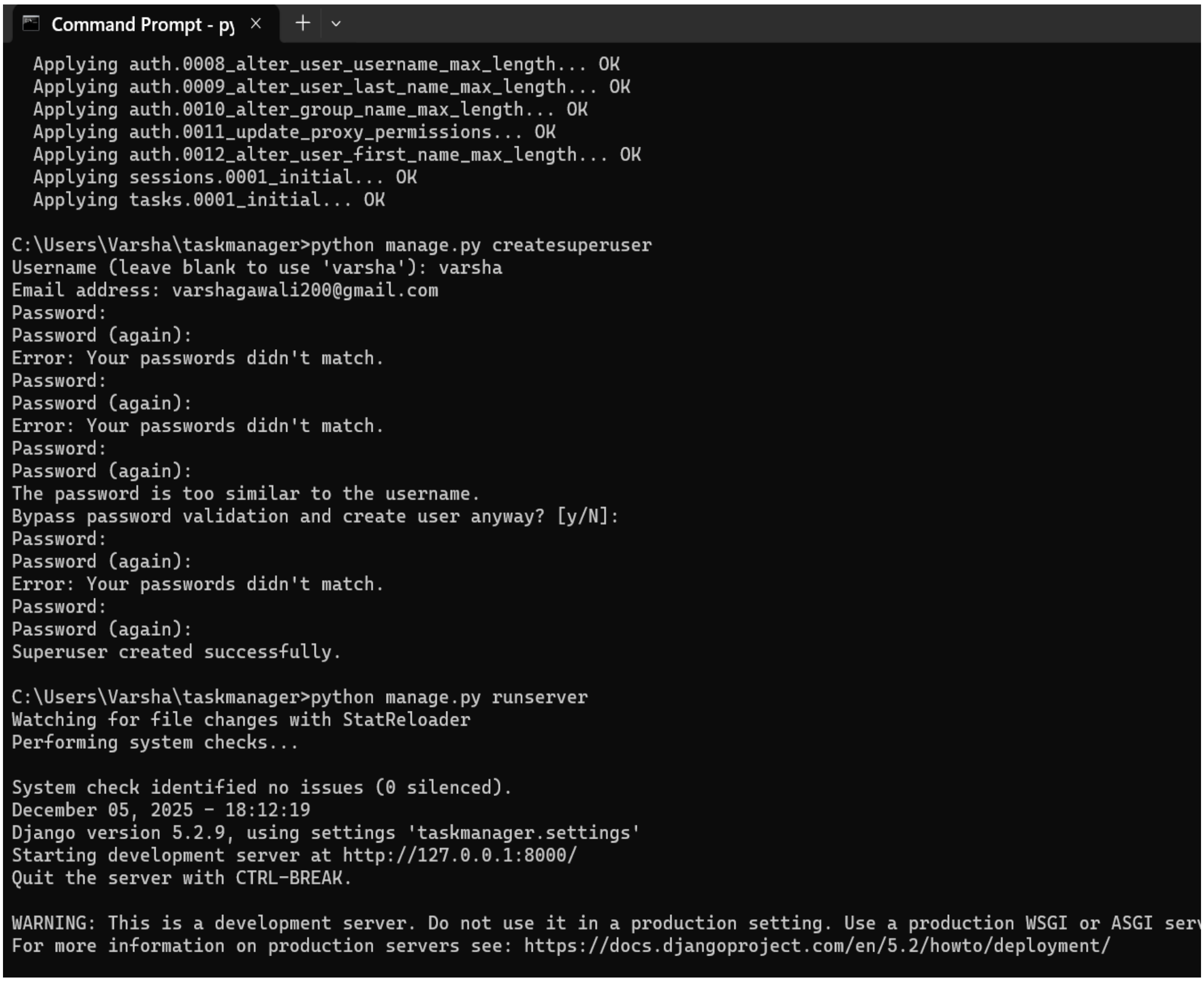
```
    path('', include(router.urls)),
]
```

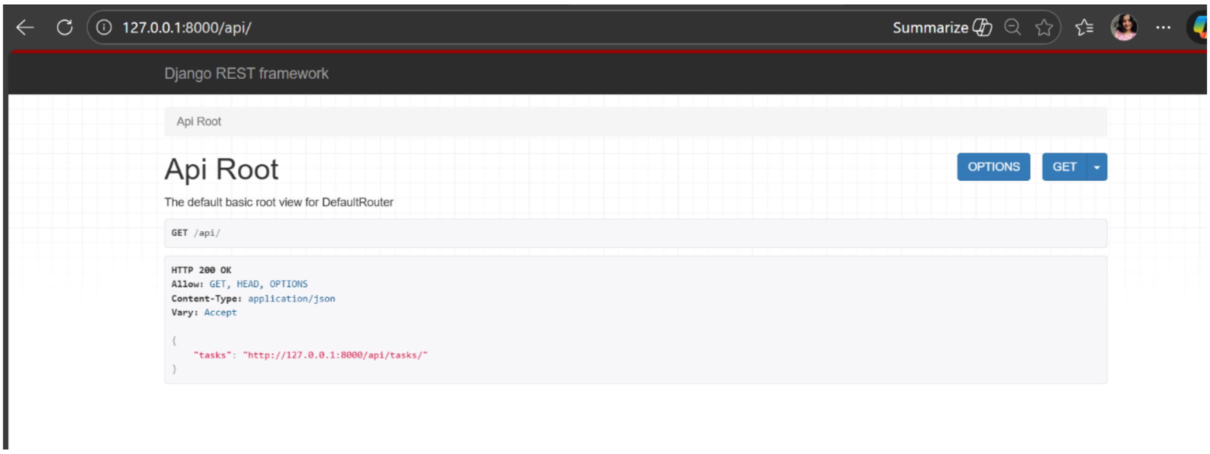
9. Include app URLs in project (taskmanager/urls.py)

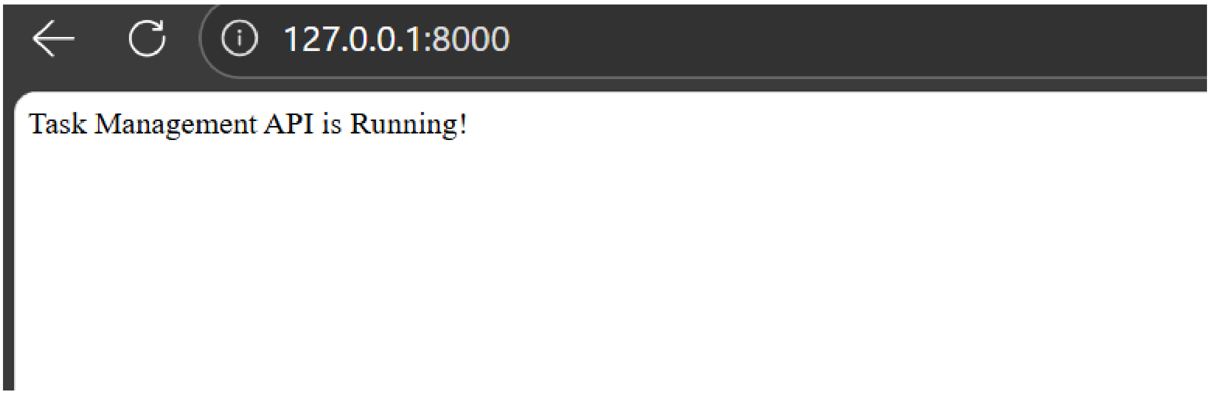
```
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('tasks.urls')),
]
```

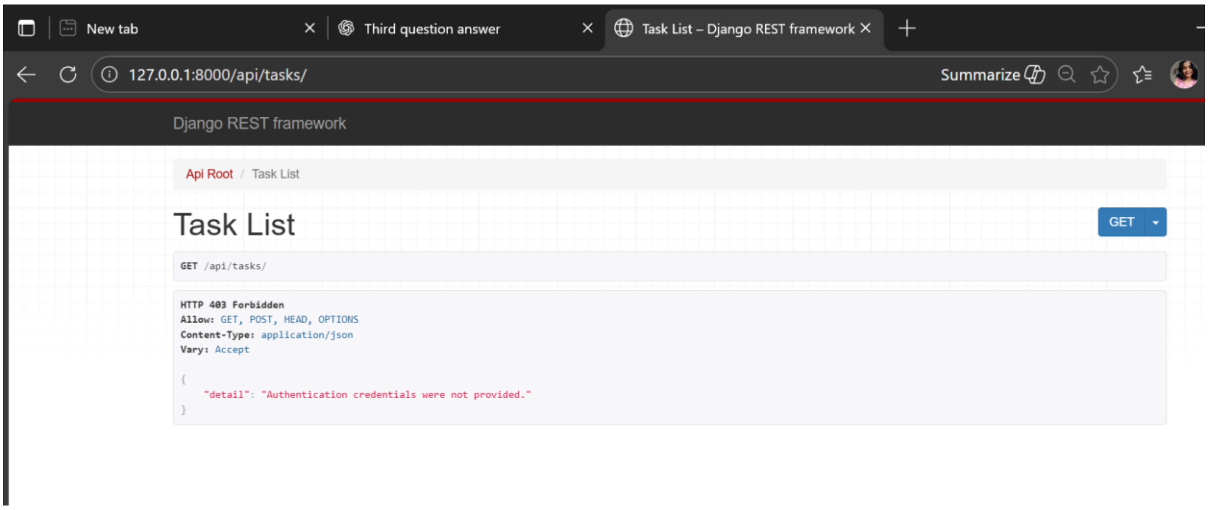
10. Create superuser & run server

```
python manage.py createsuperuser
python manage.py runserver
```




127.0.0.1:8000/api/
Summarize
Django REST framework
Api Root
Api Root
OPTIONS
GET
The default basic root view for DefaultRouter
GET /api/
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
{
"tasks": "http://127.0.0.1:8000/api/tasks/"
}


127.0.0.1:8000
Task Management API is Running!


New tab
Third question answer
Task List – Django REST framework
127.0.0.1:8000/api/tasks/
Summarize
Django REST framework
Api Root / Task List
Task List
GET
GET /api/tasks/
HTTP 403 Forbidden
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
{
"detail": "Authentication credentials were not provided."
}